NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

CCSFEN1L – INTRODUCTION TO SOFTWARE ENGINEERING

Project Documentation

GROUP INFORMATION:

PROJECT TITLE: Forensic Log Analysis Documentation

Group Name: Metaguard

Section: COM243

Project Members:

No.	Name	Assigned Role
1	Berto, James	Project Manager / Backend Developer
2	Salazar, Franz	Documentation
3	Sandoval, Christian Ace	Tester / QA
4	Santos, Tyra	Initial Designer
5	Wales, Cyralin	Frontend Developer

TABLE OF CONTENTS

CCSFEN1L – INTRODUCTION TO SOFTWARE ENGINEERING
Project Documentation 1
1. PROJECT OVERVIEW 3
2. PROBLEM STATEMENT 3
3. PROJECT OBJECTIVES 4
3.1 General Objective 4
3.2 Specific Objectives 4
4. SCOPE AND LIMITATIONS 5
4.1 Scope 5
4.2 Limitations 5
5. REQUIREMENTS 6
5.1 Target Users and Stakeholders 6
5.2 Functional Requirements 7
5.3 Non-Functional Requirements 8
5.4 Business Rules 9
5.5 Requirements Gathering 10
6. SYSTEM MODELING AND DESIGN 12
6.1 Use Case Diagram 12
6.2 Activity Diagram 15
6.3 Class Diagram 17
6.4 Other UML Diagrams 19
7. SYSTEM ARCHITECTURE 24
8. DATABASE DESIGN 28
8.1 Entity Relationship Diagram 28
9. USER INTERFACE DESIGN 29
9.1 Login Page 29
9.2 Dashboard 29
9.3 Transaction Page 34
9.4 Other Major Interfaces 36
10. SYSTEM IMPLEMENTATION 36
10.1 Development Tools and Technologies 37
10.2 Major System Features 38
11. TRANSACTION PROCESSING 38
12. SOFTWARE TESTING 52
12.1 Test Plan 52
12.2 Test Cases and Results 52
12.3 Defects and Corrective Actions 54
12.4 User Acceptance Testing (UAT) 54
13. CHALLENGES AND SOLUTIONS 55
14. CONCLUSION 55

1. PROJECT OVERVIEW

Case Vault: Forensic Log Analysis Documentation is a secure web-based archive designed to store, organize, and retrieve analyzed forensic data. It addresses the problem of scattered and unstructured forensic records stored in shared cloud folders and spreadsheet files, which can make case documentation difficult to manage and increase the risk of unauthorized modifications or data loss.

The system is intended for digital forensics investigators, data analysts, and security analysts working in corporate or institutional settings. It provides a centralized platform for managing documentation of physical and digital evidence, digital artifacts, and forensic logs. Through authentication, role-based access control, and automated tamper detection, Case Vault aims to protect submitted records, improve data organization, and maintain an auditable history of activities. The system focuses on securely preserving and retrieving forensic data that has already been analyzed and does not perform forensic analysis itself.

2. PROBLEM STATEMENT

Managing analyzed forensic data presents challenges when case documentation is stored across scattered and unstructured platforms, such as physical documents and records, shared cloud folders, shared drives, and spreadsheet files. These practices create several organizational and security problems:

1. **Risk of Evidence Tampering:** Without proper file protection and automated tamper detection, submitted forensic documents may be vulnerable to unauthorized modifications. This can raise concerns about the integrity and reliability of stored forensic records.
2. **Lack of Auditability and Traceability:** Without a centralized and comprehensive audit trail, tracking who submitted, accessed, or modified forensic records can be difficult. This limits accountability and makes it harder to maintain a reliable history of activities related to each case.
3. **Disorganized Case Management:** Storing forensic documentation across multiple platforms makes it difficult to organize records and retrieve relevant information efficiently. Investigators and analysts may spend additional time locating case files, identifying evidence types, and tracking submission dates and case statuses.

4. **Access control concerns:** Shared storage may not distinguish what analysts, reviewers, and administrators are allowed to do.

To address these problems, Case Vault will provide a centralized archive with structured case organization, role-based access control, and automated tamper detection. These features aim to strengthen data integrity, improve accountability, and make analyzed forensic documentation easier to manage and retrieve.

3. PROJECT OBJECTIVES

3.1 General Objective

To develop Case Vault, a secure web-based archive that centralizes the submission, storage, organization, and retrieval of analyzed forensic case data while maintaining data integrity through authentication, role-based access control, automated tamper detection, and auditable activity records.

3.2 Specific Objectives

1. To provide a centralized archive for organizing analyzed forensic records according to case number, evidence type, analyst, submission date, and case status.
2. To implement secure authentication and role-based access control to ensure that only authorized users can access and manage forensic case records according to their assigned permissions.
3. To implement document protection, version control, and automated tamper detection to identify potential unauthorized modifications and preserve the history of submitted forensic documents.
4. To develop search, filtering, and audit logging functionalities that improve record retrieval, track user activities, and maintain an accountable history of actions performed within the system.

4. SCOPE AND LIMITATIONS

4.1 Scope

- **Forensic Data Archiving:** The system will provide a centralized repository for storing analyzed forensic records involving physical and digital evidence, including documentation of tampered or destroyed objects, digital artifacts, and forensic logs.
- **Secure Document Submission and Management:** The system will allow authenticated and authorized analysts to submit forensic case records with supporting PDF documentation. It will restrict direct editing of original submitted documents, preserve version history, and support controlled updates to case metadata, such as status and tags.
- **Case Organization and Retrieval:** The system will organize records by case number, evidence type, analyst, submission date, and case status. Authorized users will be able to search, view, and filter records based on available case information.
- **Security and Access Control:** The system will implement authentication and role-based access control with separate permission levels for analysts, reviewers, and administrators. It will also include automated tamper detection to flag potential unauthorized modifications to submitted documents.
- **Audit Trail and Record Management:** The system will record activities such as document uploads, record views, tamper flags, status changes, and deletions. Deletion privileges will be restricted to administrators, and records will be soft-deleted and archived rather than permanently removed to help preserve the case history.

4.2 Limitations

- **No Forensic Evidence Analysis:** The system will not perform forensic examinations, investigate cybercrime incidents, or analyze raw digital evidence. It will only store and manage forensic data that has already been analyzed.
- **PDF-Only Written Documentation:** The system will accept PDF files for written forensic documentation. Other document formats will not be accepted as written documentation under the proposed file submission requirements.
- **Restricted Modification and Deletion:** Original submitted documents cannot be directly edited by analysts. Changes requiring a revised document will be stored as

separate versions linked to the original record. Only authorized users may update permitted metadata, while deletion is restricted to administrators and uses soft deletion rather than permanent removal.

- **Access Limited by User Permissions:** Users will only be able to perform actions allowed by their assigned roles. Analysts, reviewers, and administrators will have different permission levels, limiting access to functions outside their authorization.
- **Dependence on Submitted Information:** The system will serve as an archive for submitted forensic documentation. The accuracy and completeness of the information provided will depend on the data and documents submitted by authorized users.

5. REQUIREMENTS

5.1 Target Users and Stakeholders

User/Stakeholder	Description
Digital Forensics Investigators	Primary users responsible for handling, analyzing, and submitting analyzed case data into the system
Data Analysts	Users who analyze forensic data and submit completed case documentation into the secure archive.
Security Analysts	Internal/institutional users in corporate or institutional settings who retrieve, view, and utilize analyzed forensic records.
Reviewers	Stakeholders with specific permissions under Role-Based Access Control to review case data.
Administrator	System managers are authorized to provision credentials (email and password) for analyst users, manage system settings, and perform restricted soft-deletions/archiving of records.

5.2 Functional Requirements

User/Stakeholder	Description
Administrator, Analyst, Reviewer (FR-01)	The system shall authenticate a user with an email address and password and return a bearer access token when the credentials are valid.
Administrator, Analyst, Reviewer (FR-02)	The system shall reject invalid or expired access tokens and deny access to deactivated user accounts.
Administrator, Analyst, Reviewer (FR-03)	The system allows each signed-in user to use only the functions permitted for their role.
Administrator (FR-04)	An administrator can create user accounts, view the user list, change a user's role or active status, and see the roles available in the system.
Administrator, Analyst (FR-05)	The system shall allow an authorized user to create a case with a unique case number and case type.
Administrator, Analyst, Reviewer (FR-06)	The system shall allow an authorized user to view and retrieve cases and filter the case list by case number, case type, or status.
Administrator, Reviewer (FR-07)	An authorized administrator or reviewer can change a case's status. The system records the updated status on the case.
Administrator (FR-08)	An administrator can remove a case from active use. The case record remains stored, but is marked as deleted.
Administrator, Analyst (FR-09)	An authorized administrator or analyst can add a PDF evidence document to an existing case that has not been deleted.
Administrator, Analyst (FR-10)	When an authorized user replaces an evidence document, the system keeps the earlier version and adds the replacement as a new version.
Administrator, Analyst, Reviewer (FR-11)	An authorized user can view the evidence attached to a case, review its available versions, and download a selected version.

Administrator, Analyst, Reviewer (FR-12)	The system shall allow an authorized user to verify the integrity of a stored evidence version and flag a detected hash mismatch. Available to roles with evidence:view through the version verification endpoint, which compares SHA-256 hashes when requested.
Administrator, Analyst, Reviewer (FR-13)	Authorized users can create and view tags, then add tags to cases, view their assigned tags, or remove them.
Administrator (FR-14)	An administrator can view audit records and narrow the results by case, evidence document, or the user who performed an action.
System (FR-15)	The system records selected account, case, evidence, tag, and integrity-check actions with the responsible user and the time of the action. The current record does not cover every read or change.
Analyst,Reviewer (FR-16)	The system shall allow authorized users to search and filter records by analyst and submission date. Implementation: This is stated in the proposal but is not available in the supplied backend; current case filters are case number, case type, and status.

5.3 Non-Functional Requirements

Performance	Requirement
Security	Passwords are hashed with bcrypt; access tokens are signed with HS256; protected routes require authentication and permission checks. Database URI and JWT secret are configuration values and must be protected. No independent security assessment was supplied.
Usability	FastAPI interactive API documentation is available for API use. A user-facing frontend, usability target, and usability evaluation were not supplied.

Reliability	Database initialization retries up to six times, with a three-second delay. Case creation, user creation, and initial evidence metadata plus its audit record use MongoDB transactions. Other writes are not uniformly transactional. No uptime measurement was supplied.
Performance	The application uses asynchronous FastAPI and Motor operations. No response-time target, concurrency target, load test, or performance measurement was supplied.
Data Integrity	Data Integrity The system shall preserve original submitted documents, flag potential unauthorized modifications using hash verification, and maintain linked document versions to support record traceability.
Maintainability	The system shall use a modular design that separates authentication, case management, document storage, and audit logging to support independent development and testing.

5.4 Business Rules

1. Users must authenticate before using protected endpoints. Inactive accounts are denied access.
2. Permissions are granted through role-permission mappings. The seed service creates these roles and grants:

Permission	Admin	Analyst	Reviewer
case: create	Yes	Yes	No
case: view	Yes	Yes	Yes
case: update_status	Yes	Yes	No
case: delete	Yes	No	No
evidence: upload	Yes	Yes	No
evidence: view	Yes	Yes	Yes

user: image	Yes	No	No

3. There is no public registration endpoint. An administrator creates user accounts.
4. Case numbers, user emails, role names, permission codes, and tag names are unique according to the models/database indexes.
5. Evidence uploads must declare application/pdf and must refer to an existing case that has not been soft-deleted.
6. Adding a replacement file creates an additional EvidenceVersion; prior version records are not overwritten. The evidence record points to the current version.
7. Integrity checks are performed when the verify endpoint is called. A mismatch is flagged and logged; files are not monitored continuously.
8. Only a user with case:delete can soft-delete a case. Soft-deleted cases are excluded from ordinary case listing and cannot be retrieved through the case endpoint or used for new evidence uploads.
9. Audit listing requires user:manage, which the seed service grants only to admin.
10. The API does not enforce a fixed list or ordered progression of case status values. Users with case:update_status can submit a status string.
11. Audit records are not a complete log of all reads/actions, and audit/data changes are not atomic for every endpoint.

5.5 Requirements Gathering

☐ *Interview*

☐ *Observation*

☐ *Questionnaire*

☑ *Document Analysis*

☐ *Other:* _____

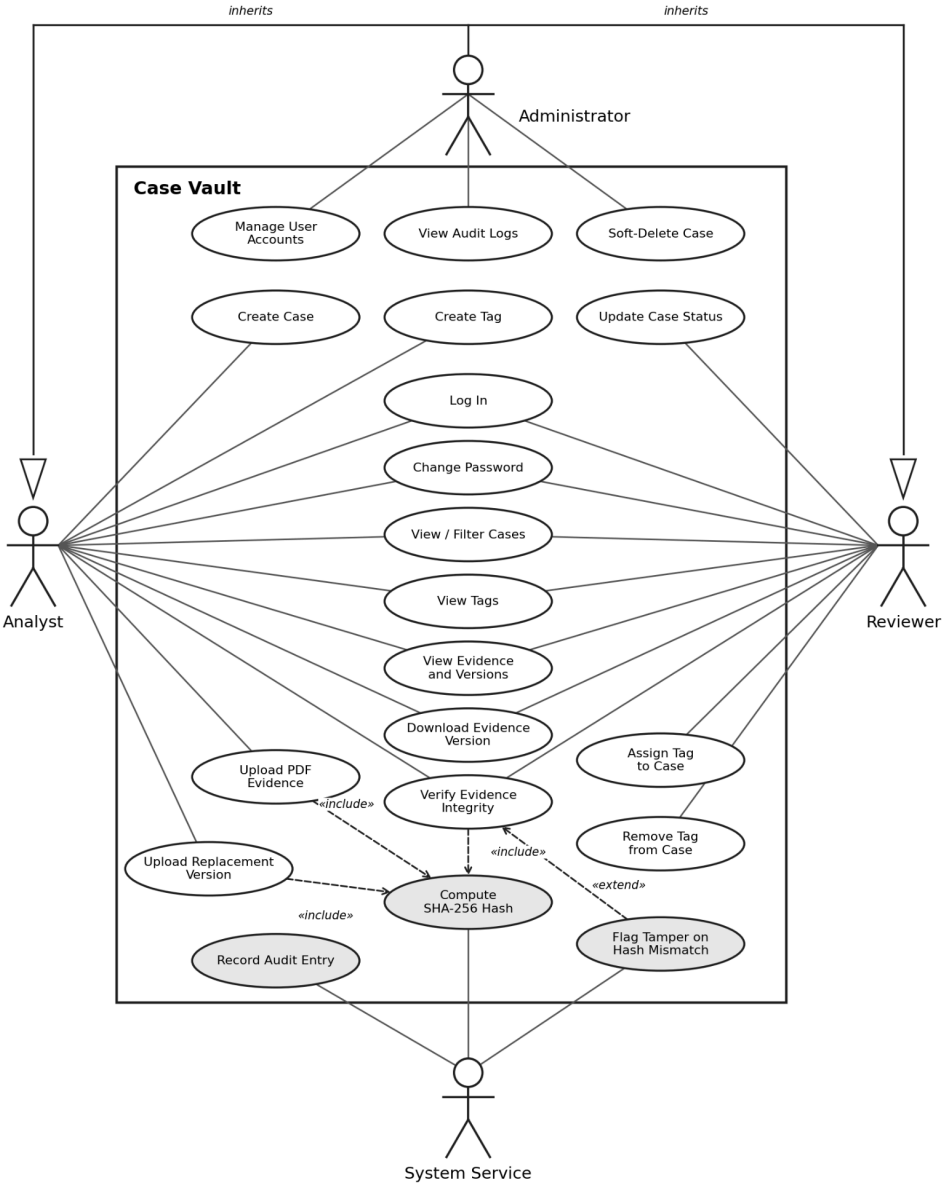
Brief Description:

Requirements were gathered by analyzing the supplied Case Vault project proposal and comparing those statements with the supplied documentation format, README, dependency manifest, and backend source code. The source review covered routers, schemas, models, authentication/permission dependencies, database configuration, and services. The Nodeguard documentation was considered only as an example of presentation depth; it describes another system and was not treated as evidence of Case Vault requirements or implementation. No interview, observation, questionnaire, or stakeholder validation records were supplied.

6. SYSTEM MODELING AND DESIGN

6.1 Use Case Diagram

Figure 1. Use Case Diagram



Description:

Use Case Diagram

Shows interactions between the system actors and the Case Vault boundary. The system defines four actors:

Analyst. A user who creates cases, uploads PDF evidence and replacement versions, creates tags, and views and downloads stored evidence.

Reviewer. A user who reviews case data, updates case status, assigns or removes tags, and checks evidence integrity. Reviewers cannot upload evidence.

Administrator. An authenticated user who inherits all Analyst and Reviewer functions (shown by the two “inherits” generalizations). Administrators also manage user accounts, view audit logs, and soft-delete cases.

System Service. An automated background actor that computes SHA-256 hashes, flags tampering, and records audit entries.

Key use cases:

Log In. Authenticates staff with email and password and returns a signed JWT bearer token. It is available to all roles. There is no public registration.

Change Password. Users can change their own password.

Manage User Accounts. Allows administrators manage user permissions and creation.

View Audit Logs. Allow administrators to view audit records, filtered by case, evidence document, or acting user.

Create Case. Allows an analyst or administrator to register a case with a unique case number and case type.

View / Filter Cases. Allow users to filter and view cases using their case number, type, or status.

Update Case Status. Allows a reviewer or administrator to change a case’s status, viewable in the audit log.

Soft-Delete Case. Allows an admin to mark a case as deleted. The record would only be viewable by the administrator.

Create Tag, View Tags, Assign Tag to Case, Remove Tag from Case. Tag management. Analysts create tags, while reviewers and administrators assign and remove them on cases.

Upload PDF Evidence. Allows analysts and admins to attach a PDF to an active case. It includes computing the file’s SHA-256 hash.

Upload Replacement Version. It adds a new version of an evidence file without overwriting earlier versions.

View Evidence and Versions. Allow all roles to list the evidence on a case and its version history.

Download Evidence Version. Allow all roles to download a selected stored version.

Verify Evidence Integrity. Recomputes the SHA-256 hash of a stored version on demand and compares it with the hash recorded at upload.

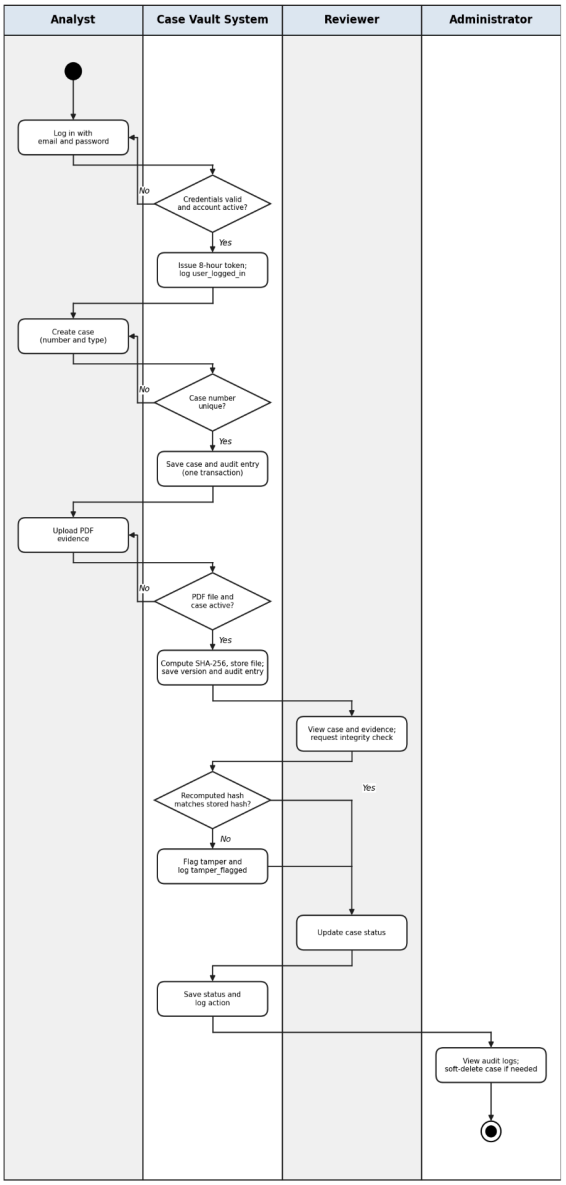
Compute SHA-256 Hash. An automated function, it is included by evidence upload, replacement upload, and integrity verification.

Flag Tamper on Hash Mismatch. An extension of Verify Evidence Integrity. Only runs when the hashes differ, setting the tamper flag and writing an audit event.

Record Audit Entry. A function that stores the actor, action, and time, with optional case and evidence context.

6.2 Activity Diagram

Figure 2. Activity Diagram



Description:

It demonstrates how Case Vault processes a user's request from its login to logout. It shows the two parties: the system itself and the user/s (Administrator, Analyst, and Reviewers).

The user opens up the system and enters an email address and password. After signing in, the user selects an action to perform with each action limited to what the user's role is and will allow. Example being:

- **Administrators** can do everything, including managing users, soft-deleting cases, and viewing audit logs.
- **Analysts** can create cases and upload and view evidence.
- **Reviewers** can view cases and evidence and update case status.

When the result is shown, the user can select another action or end the session.

As for the **System** itself, it checks that the credentials entered are valid and the account is active.

- If the check fails, it will show an error of HTTP (401 or 403) and returns to the login step.
- If it passes, it issues and generates an 8-hour access token and logs a user_logged_in audit entry.
- For each and every action made by users, it checks whether the current user has the correct and required authorization. If not it'll show "Access denied" (403) and return to action selection.
- As for actions that have the required and correct authorization it will continue and record the action in the audit log.

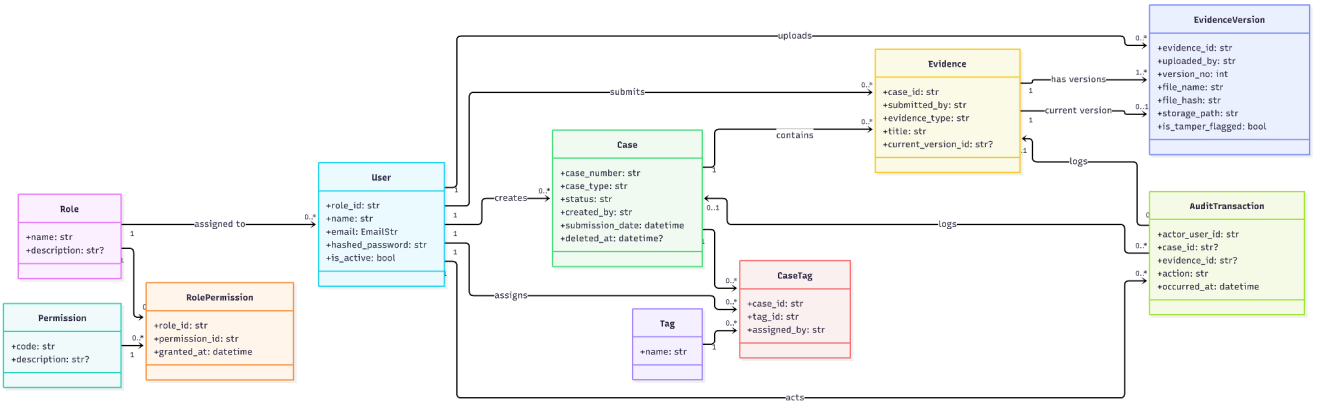
Actions. Once permission is confirmed, the system branches on the action type:

- **Create case.** It checks that the case number is unique. If it already exists, the system shows a 409 error. Otherwise, it saves the case and its audit entry in one transaction.
- **Upload evidence.** The system checks that the file is a PDF and that the case exists and is not deleted. If either check fails, it rejects the upload (400 or 404). If both pass, it computes the file's SHA-256 hash and stores the file in GridFS. It then saves a new evidence version with the hash and an audit entry.
- **Verify evidence.** The system recomputes the SHA-256 hash of the stored file and compares it to the stored hash. The result depends on the comparison:
    - **If the hashes match:** the system reports that the evidence is intact.
    - **If the hashes do not match:** the system sets the tamper flag on that version and logs tamper_flagged.
- **Update status or tags.** The system saves the change and logs the action.
- **Administrator tasks.** The administrator manages user accounts, soft-deletes a case (it is marked deleted but kept in the database), or views the audit logs.

All branches rejoin at one point where the result is shown to the user.

6.3 Class Diagram

Figure 3. Class Diagram



Description:

Figure 3 outlines the core data structures and relationships in Case Vault, comprising ten classes. The diagram details their attributes and associations.

Role. Defines the type of user account: Administrator, Analyst, or Reviewer. Stores the role name and an optional description.

- One role can be assigned to zero or many users (1 to 0..*).

Permission. Represents an allowed action, such as case:create or evidence:upload. Stores a unique permission code and an optional description.

RolePermission. Connects roles to permissions, specifying which actions each role can perform. Stores the role ID, permission ID, and the date granted.

- A role can hold many permissions, and a permission can belong to many roles (1 to 0..* on each side).

User. Represents system staff. Stores the role ID, name, email, hashed password, and active status. Administrators can deactivate accounts. Depending on their role, users may create cases, submit evidence, upload evidence versions, assign tags, and perform logged actions.

- Each of these is a one-to-many relationship (1 to 0..*).

Case. Represents a forensic case record. Stores the case number, type, status, creator, submission date, and deleted date. The deleted date indicates a soft-deleted case, which is hidden from normal use but retained in the database.

- One case contains zero or many pieces of evidence (1 to 0..*).

Evidence. Represents an item of evidence within a case. Stores the case ID, submitter, evidence type, title, and current version ID.

- Each evidence record has one or more versions (1 to 1..*) and points to at most one current version (1 to 0..1).

EvidenceVersion. Represents a single uploaded PDF file for an evidence record. Stores the evidence ID, uploader, version number, file name, SHA-256 file hash, storage path, and a tamper flag. Replacing a document creates a new version; previous versions remain unchanged. The tamper flag is set if a verification check finds the file's recomputed hash does not match the stored hash.

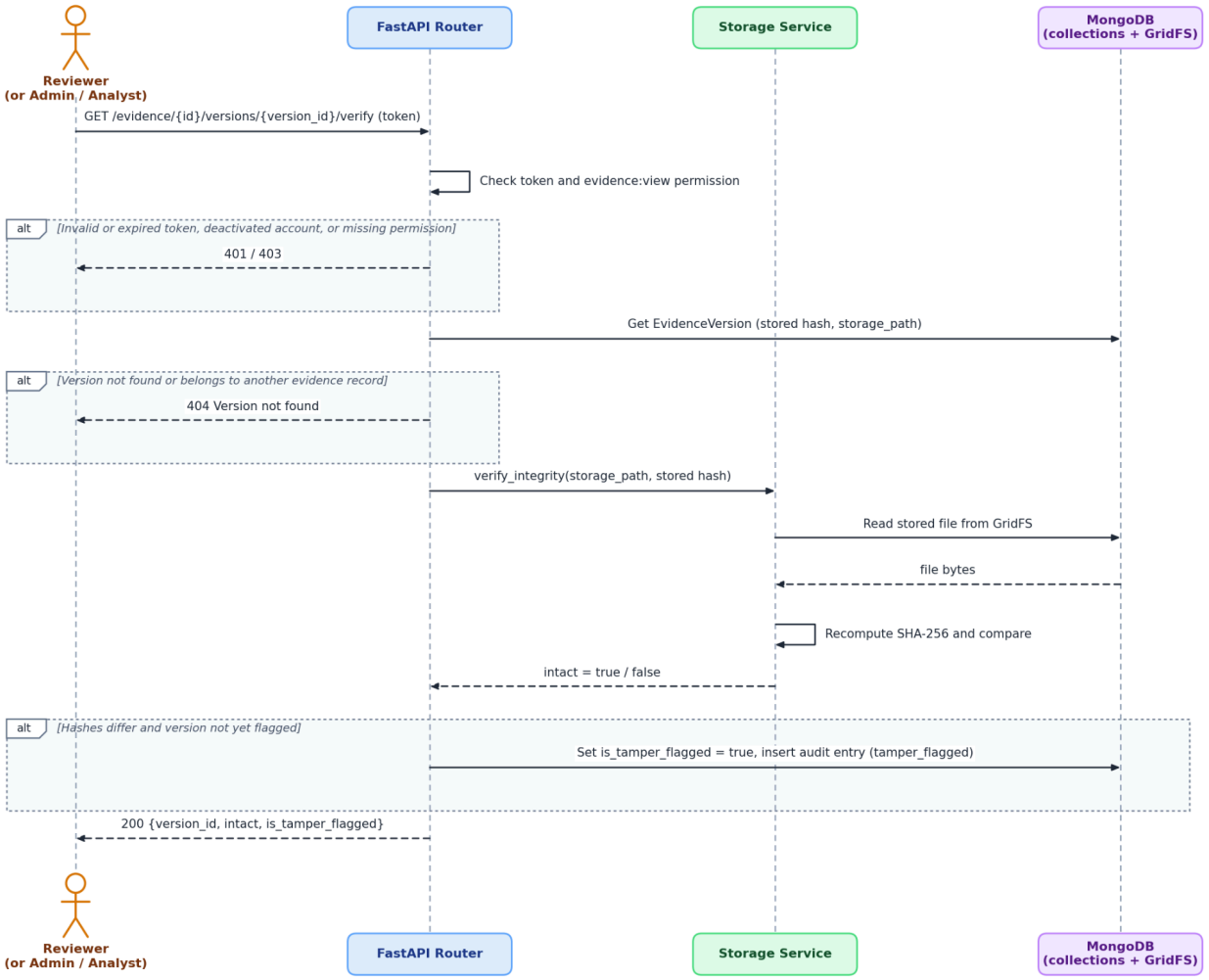
Tag. Represents a label that can be attached to cases. It stores a unique tag name.

CaseTag. Links tags to cases. It stores the case ID, tag ID, and the user who assigned the tag. A case can have many tags, and a tag can be used on many cases (1 to 0..* on each side).

AuditTransaction. Records selected system actions, including account, case, evidence, tag, and integrity checks. Stores the acting user, related case and evidence (both optional), action type, and timestamp. A case or evidence record may have multiple audit entries, each linked to the acting user.

6.4 Other UML Diagrams

Sequence Diagram (verify sequence)

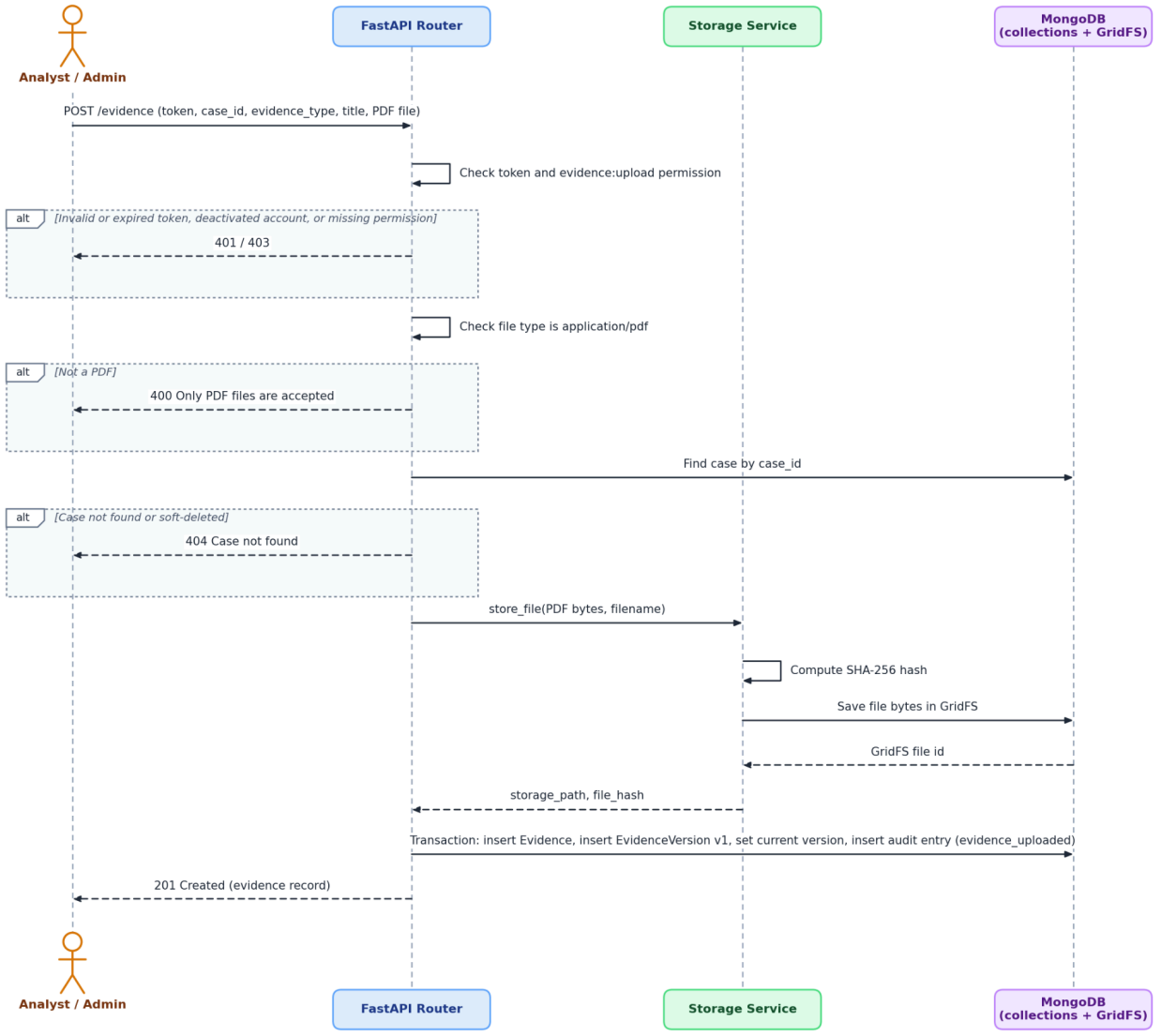


Description:
The diagram shows the order of messages when a user checks the integrity of a stored evidence version. The participants are the user, the FastAPI Router, the Storage Service, and MongoDB.

1. **Submit request:** The user sends GET /evidence/{id}/versions/{version_id}/verify with a token.
2. **Check access:** The router checks the token and the evidence:view permission. An invalid or expired token, a deactivated account, or a missing permission returns 401 or 403.
3. **Get version record:** The router retrieves the EvidenceVersion from MongoDB, including its stored hash and storage_path. If the version is not found or belongs to another evidence record, it returns 404 “Version not found”.

4. **Verify file:** The router sends the storage_path and stored hash to the Storage Service. The service reads the file bytes from GridFS, recomputes the SHA-256 hash, and compares it with the stored hash. It returns intact = true or false to the router.
5. **Flag tampering:** If the hashes differ and the version has not yet been flagged, the router sets is_tamper_flagged = true and inserts a tamper_flagged audit entry. A version that is already flagged is not flagged or logged again.
6. **Confirm:** The router returns 200 with the version_id, intact, and is_tamper_flagged.

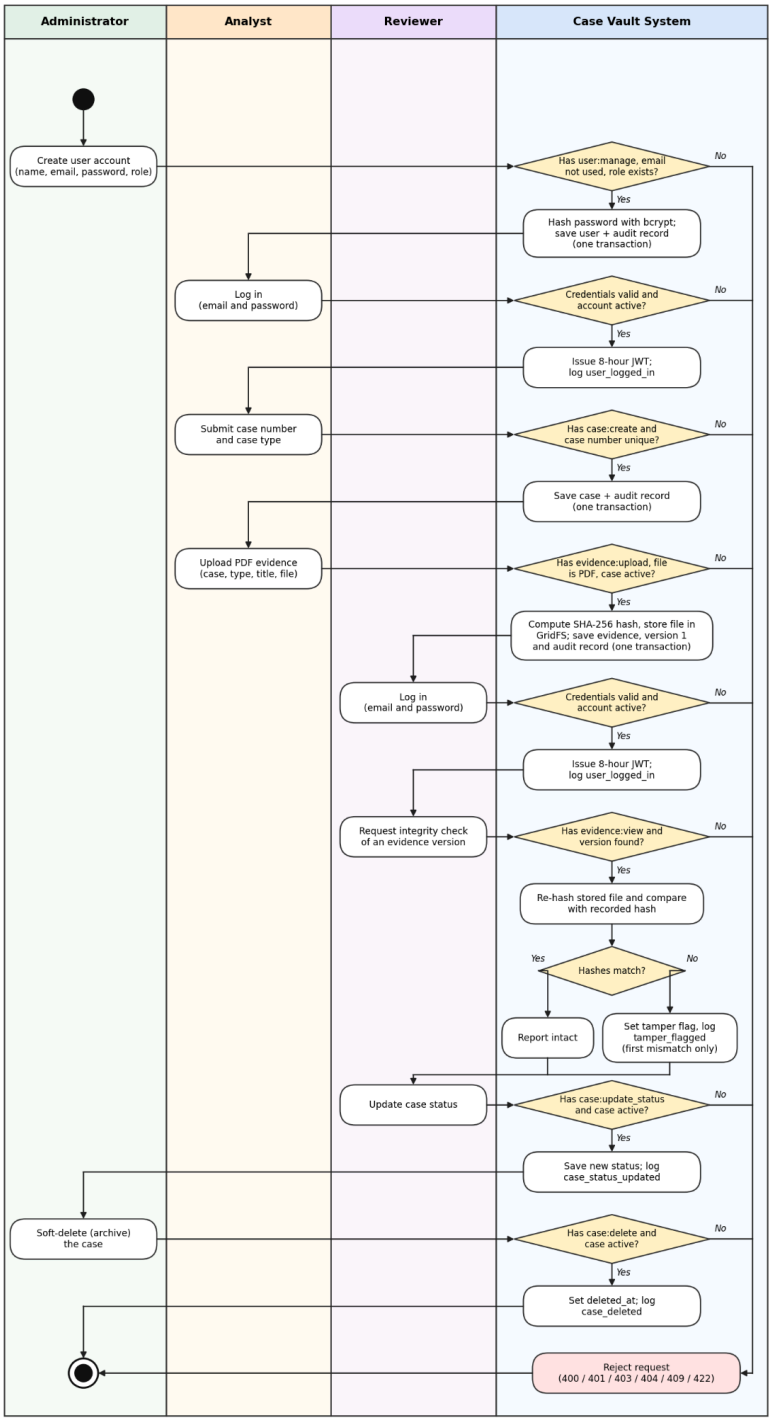
Sequence Diagram (Upload sequence)



Description:
The diagram shows the order of messages when an Analyst or Admin uploads a PDF evidence file. The participants are the Analyst/Admin, the FastAPI Router, the Storage Service, and MongoDB (collections and GridFS).

1. **Submit request:** The user sends POST /evidence with a token, case ID, evidence type, title, and PDF file.
2. **Check access:** The router checks the token and the evidence:upload permission. An invalid or expired token, a deactivated account, or a missing permission returns 401 or 403.
3. **Check file type:** The router checks that the file type is application/pdf. If not, it returns 400 “Only PDF files are accepted”.
4. **Check case:** The router looks up the case by case ID in MongoDB. If the case is not found or has been soft-deleted, it returns 404 “Case not found”.
5. **Store file:** The router sends the PDF bytes and filename to the Storage Service. The service computes the SHA-256 hash, saves the file bytes in GridFS, and receives a GridFS file ID. It returns the storage_path and file_hash to the router.
6. **Save records:** The router runs one transaction that inserts the Evidence record, inserts EvidenceVersion v1, sets it as the current version, and inserts the evidence_uploaded audit entry.
7. **Confirm:** The router returns 201 Created with the evidence record.

State Diagram



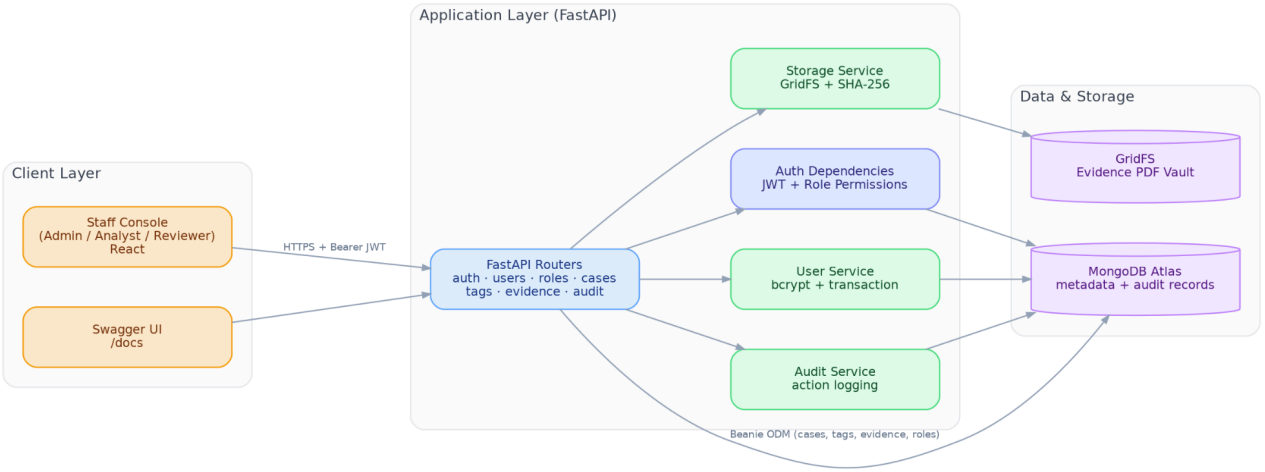
Description:

This state diagram presents the main workflow of our Case Vault using a swimlane diagram with four lanes: Administrator, Analyst, Reviewer, and the Case Vault System. Each action is sent to the system, which verifies permissions and relevant conditions prior to processing. If

any checkfails, the system rejects the request and returns an error code (400, 401, 403, 404, 409, or 422), terminating the syntax. The main steps are:

1. **Create user account:** The administrator can create a user, containing their name, email, password, and role. The system service checks the user:manage permission, checks if the email is unused, and if that role exists. It then hashes the password with bcrypt and saves both the user and an audit record in one transaction.
2. **Log in:** The analyst (and later the reviewer) submits an email and password. If the credentials are valid, and active, the system issues an 8-hour token and logs user_logged_in.
3. **Submit case:** The analyst creates or submits a case number and case type. The system service checks case:create then verifies the case number is unique and has no conflict, then saves the case and an audit record in one transaction.
4. **Upload PDF evidence:** The analyst uploads a PDF file for a case. The system then checks evidence:upload, verifies the file is a PDF and if the case is still active. It then computes a SHA-256 hash, stores the file in GridFS, and saves the evidence and its version in an audit record in one fell swoop.
5. **Integrity check:** The reviewer requests a check of an evidence version. The system checks evidence:view and that the version exists, then re-hashes the stored file and compares it with the recorded hash. If the hashes of both files match , it returns as intact. If not, it sets the tamper flag and logs tamper_flagged (first mismatch only).
6. **Update case status:** The reviewer updates the case status and the system checks case:update_status and if the case is active, then saves the new status and logs case_status_updated.
7. **Soft-delete case:** The administrator can archive a case. The system checks case:delete and that the case is active, then sets deleted_at and logs case_deleted.

Component Diagram

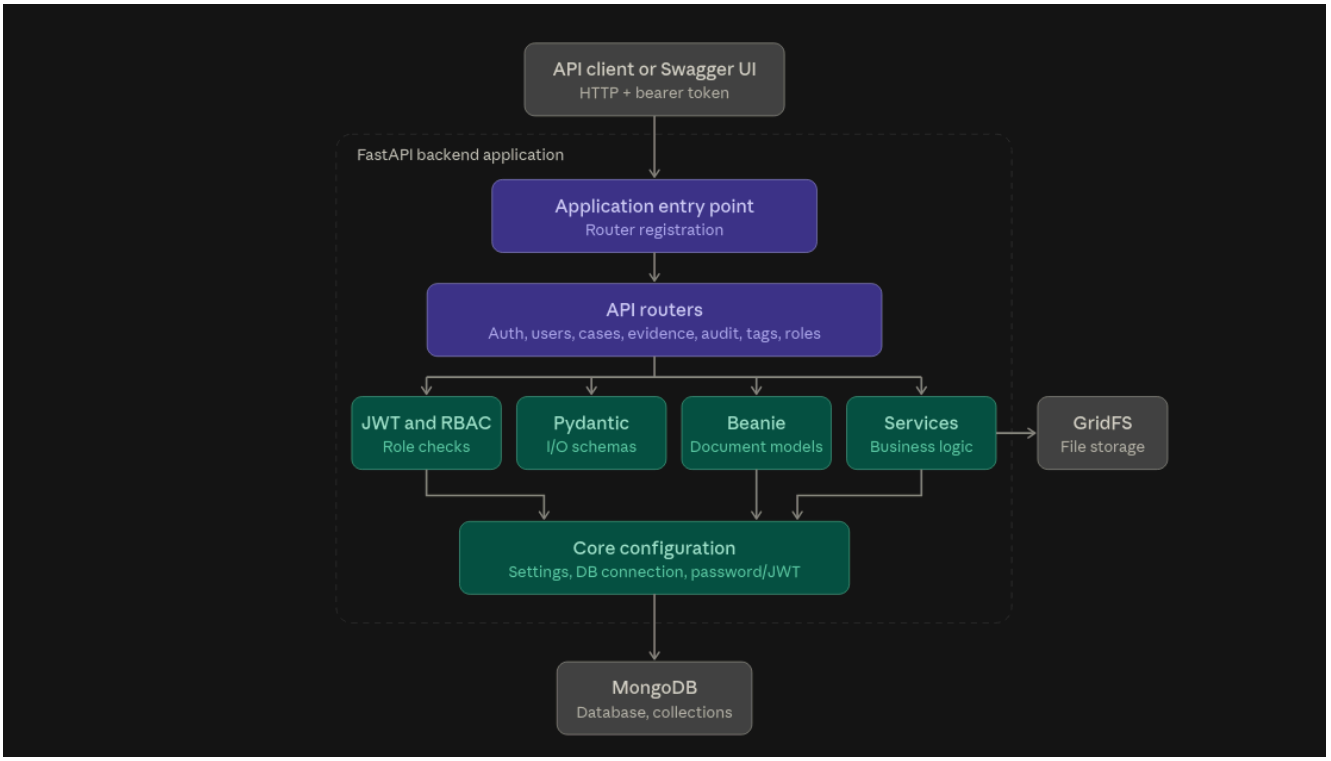


7. SYSTEM ARCHITECTURE

Figure 4. System Architecture

The Case Vault system uses a 3-tier client-server architecture. Its functions are divided into three main layers: client presentation, application business logic and evidence integrity processing, and data persistence.

Figure 4. System Architecture



Presentation Layer (Next.js / Tailwind CSS – frontend)		
Login & Dashboard	**Case Management**	**Administration**
Login page (credential authentication) Dashboard (session status, security verification indicators, system integrity overview) Navigation to main features	Case Library (search and filter of registered cases) Case Detail page (case information, evidence records, chain of custody view) Secure Submission page (case entry and PDF evidence upload)	Audit Trail page (system activity records) Access Control page (user accounts, roles, account status, permission levels) Restricted to users holding the user:manage permission

Application & Business Logic Layer (Python / FastAPI REST API – app/)	
API Routing & Security Middleware (routers/, dependencies/, core/)	**Services & Schemas (services/, schemas/)**
- REST API endpoints: routers/auth.py, users.py, roles.py, cases.py, evidence.py, tags.py, audit.py - Application entry point, router registration and /health check: main.py - Access control and JWT authentication: dependencies/auth.py (get_current_user, require_permission) - Password hashing (bcrypt) and JWT signing (HS256, 8-hour expiry): core/security.py - Environment configuration (MONGO_URI, DB_NAME, JWT_SECRET): core/config.py	- Login and role-based permission checks: routers/auth.py, dependencies/auth.py - Account creation with audit entry in one MongoDB transaction: services/user_service.py - Case creation, status updates and soft deletion: routers/cases.py - Evidence upload, version history, download and verification: routers/evidence.py - SHA-256 hashing, GridFS file storage and integrity re-check: services/storage_service.py - Audit logging helper: services/audit_service.py

- Database connection and model initialization with up to 6 retries: core/database.py - PDF-only file-type validation on upload: routers/evidence.py	- Role, permission and admin account seeding: services/seed_service.py - Request/response validation (Pydantic): schemas/

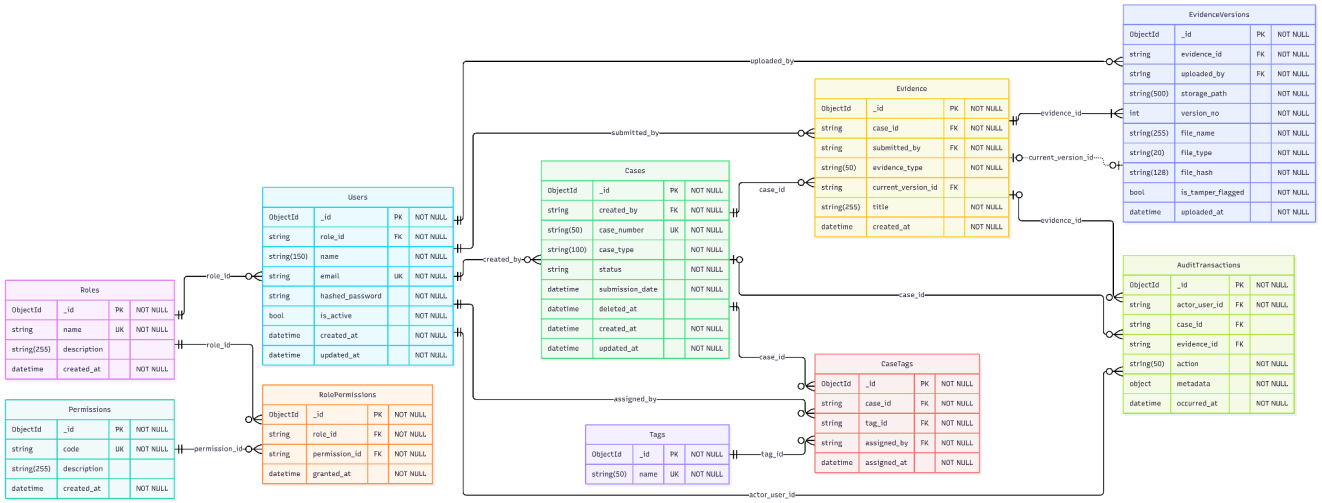
Data & Storage Layer (MongoDB Atlas / Beanie ODM & GridFS – app/models/)	
Database Collections (app/models/, app/core/database.py)	**Evidence File Storage Vault (GridFS)**
- **models/user.py:** user accounts, bcrypt password hashes, role identifier and active flag - models/role.py, permission.py, **role_permission.py:** roles (admin, analyst, reviewer), permission codes and role-to-permission mapping - **models/case.py:** case number, case type, status, creator, timestamps and soft-delete timestamp - **models/evidence.py:** evidence record, submitter, type, title and current version identifier - **models/evidence_version.py:** file name, file type, SHA-256 hash, GridFS storage identifier, tamper flag and upload time	- Uploaded PDF evidence files stored in MongoDB GridFS, in the same database as the metadata - Each file is referenced by a unique GridFS identifier, so a re-upload creates a new version and never overwrites an existing one - Stored files are re-hashed on demand and compared with the recorded SHA-256 hash to detect tampering

- models/tag.py, case_tag.py: tags and case-tag assignments - **models/audit_transaction.py:** audit records with actor, case/evidence identifiers, action, metadata and timestamp	

8. DATABASE DESIGN

8.1 Entity Relationship Diagram

Figure 5. Entity Relationship Diagram



8.2 Database Tables

Table	Purpose
user	Stores user name, unique email, hashed password, role identifier, account-active flag, and timestamps.
Roles	Stores unique role names and descriptions. Seeded roles are admin, analyst, and reviewer.
Permissions	Stores unique permission codes and descriptions.
RolePermissions	Maps role identifiers to permission identifiers.
Cases	Stores unique case number, case type, status, creator, submission/creation/update timestamps, and soft-delete timestamp.
Evidence	Stores the case identifier, submitter, evidence type, title, current-version identifier, and creation time.
EvidenceVersions	Stores evidence identifier, uploader, version number, filename, MIME type, SHA-256 hash, GridFS storage identifier, tamper flag, and upload time.
Tags	Stores unique tag names.
CaseTags	Stores case/tag association, assigning user, and assignment timestamp.
AuditTransactions	Stores actor identifier, optional case/evidence identifiers, action name, metadata, and occurrence timestamp.

9. USER INTERFACE DESIGN

9.1 Login Page

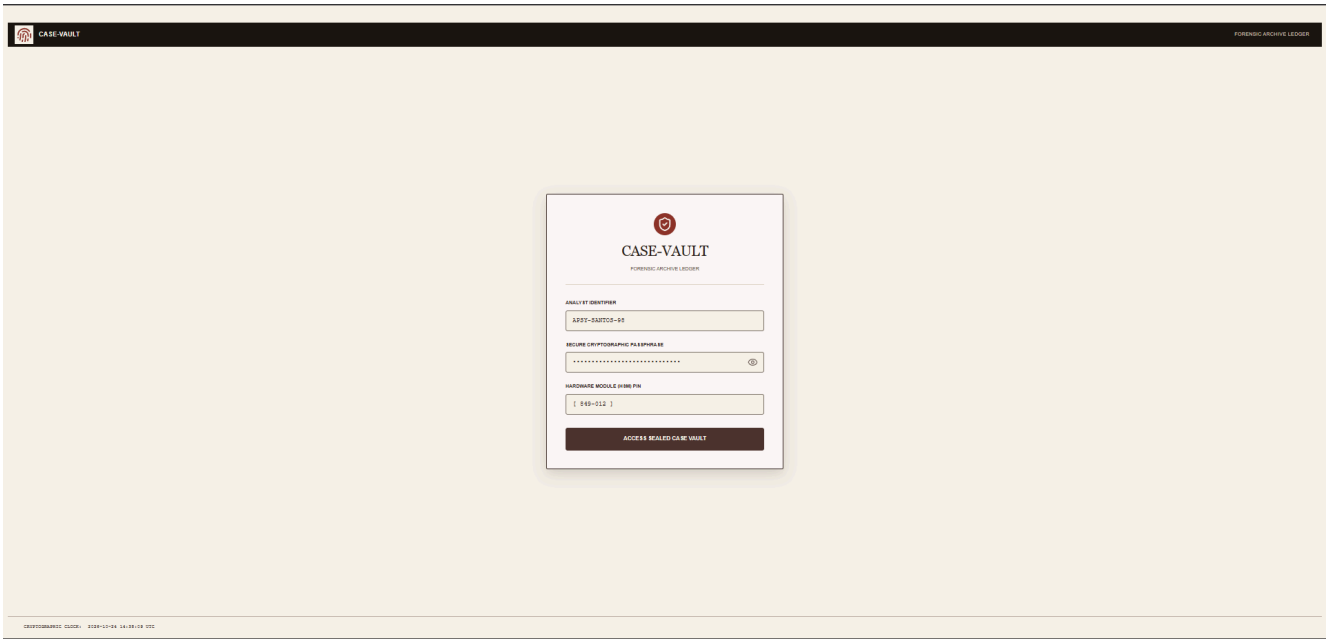


Figure 9.1. The login page provides a secure entry point to the Case Vault system, allowing authorized users to authenticate their credentials before accessing the application.

9.2 Dashboard

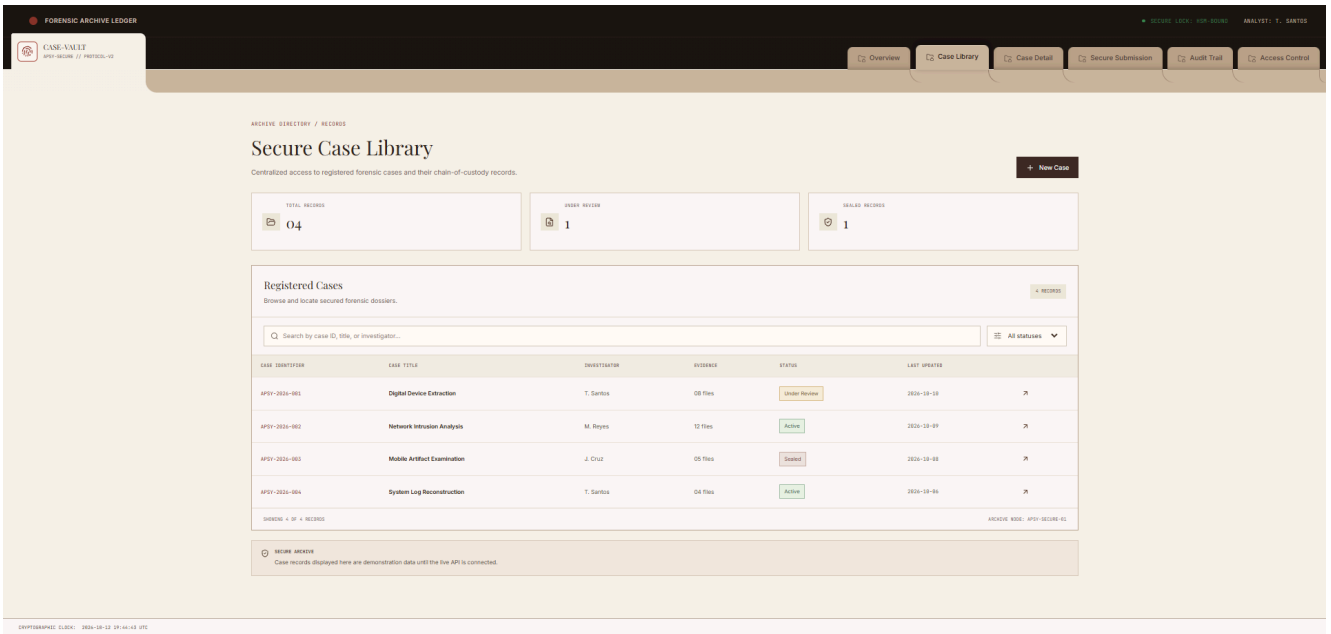


Figure 9.2.1. The dashboard provides an overview of the Case Vault system, displaying the current session status, security verification indicators, and system integrity information. It also offers direct navigation pathways to the system's main features.

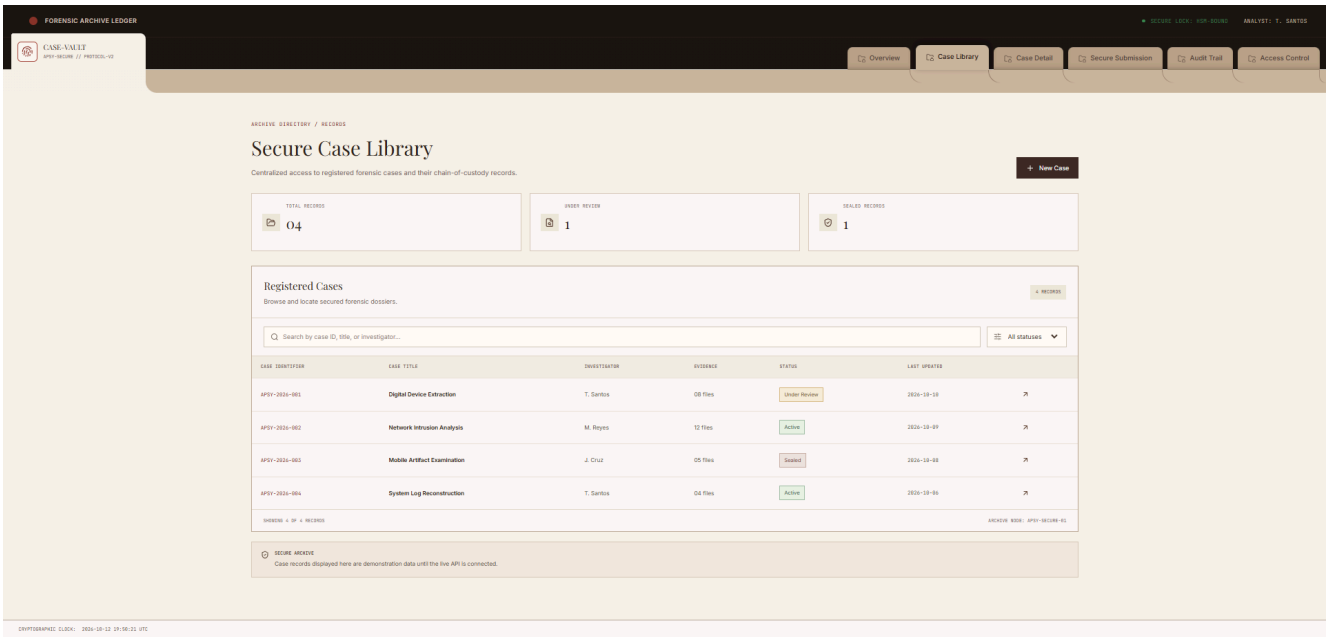
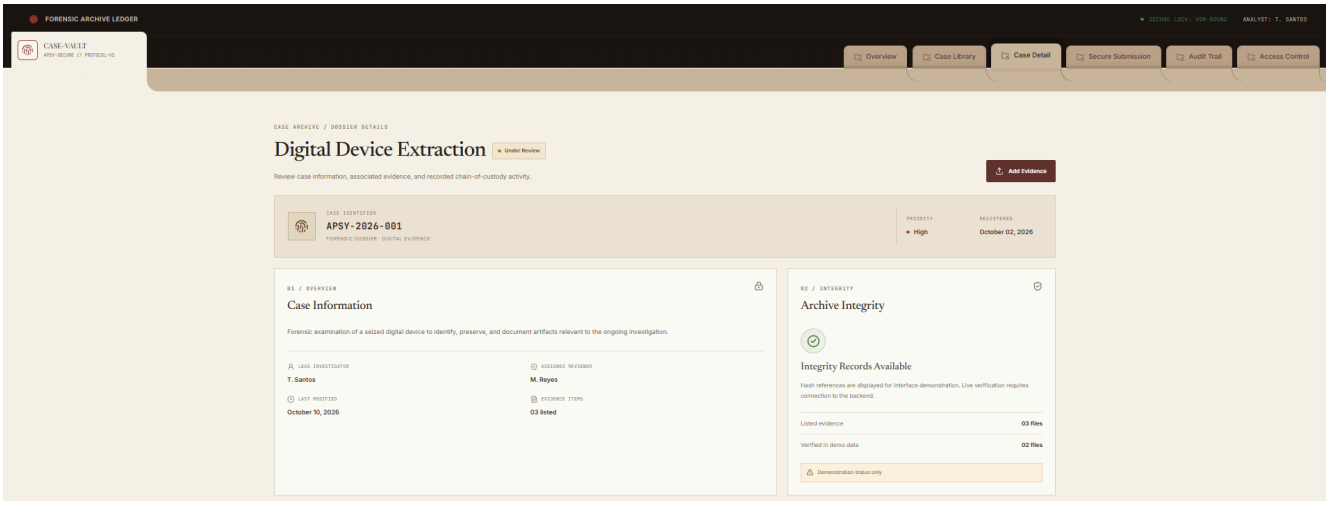


Figure 9.2.2. The Case Library displays registered forensic cases, allowing users to search, filter, and access case records.



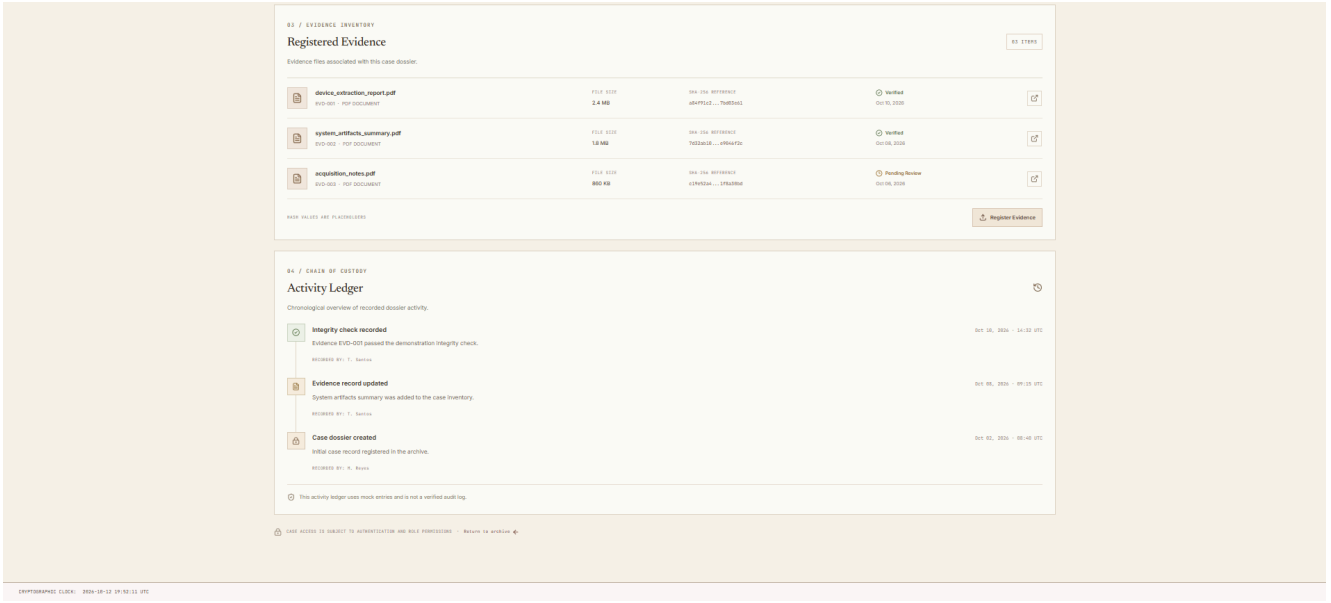


Figure 9.2.3. The Case Detail page displays information associated with a selected forensic case, including its case identifier, investigation details, and related evidence records. It supports reviewing case information and maintaining visibility over the evidence documentation and chain of custody.

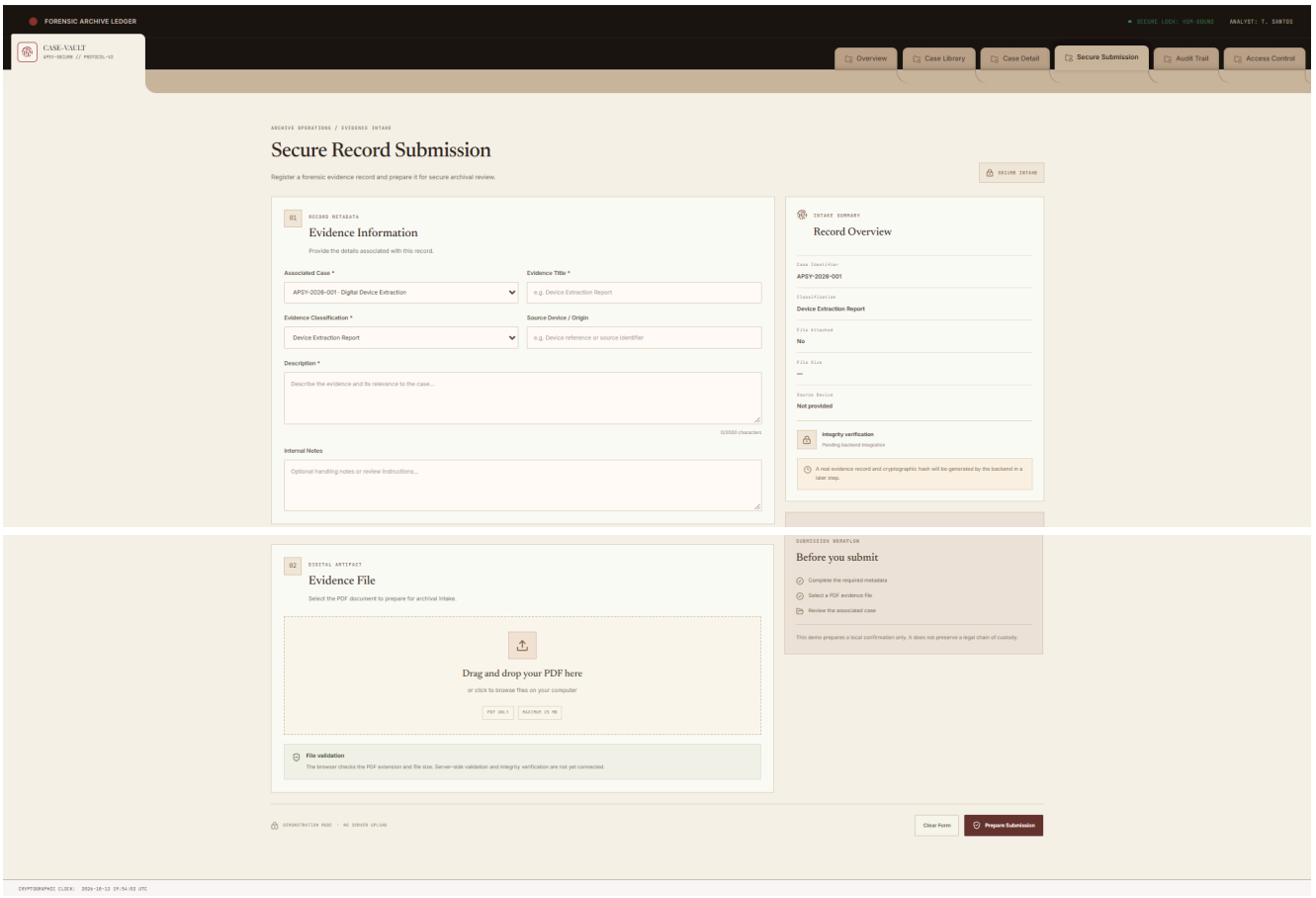


Figure 9.2.4. The Secure Submission page provides an interface for entering case information and submitting digital evidence for secure recordkeeping. It is designed to organize submitted information and support the documentation of forensic evidence.

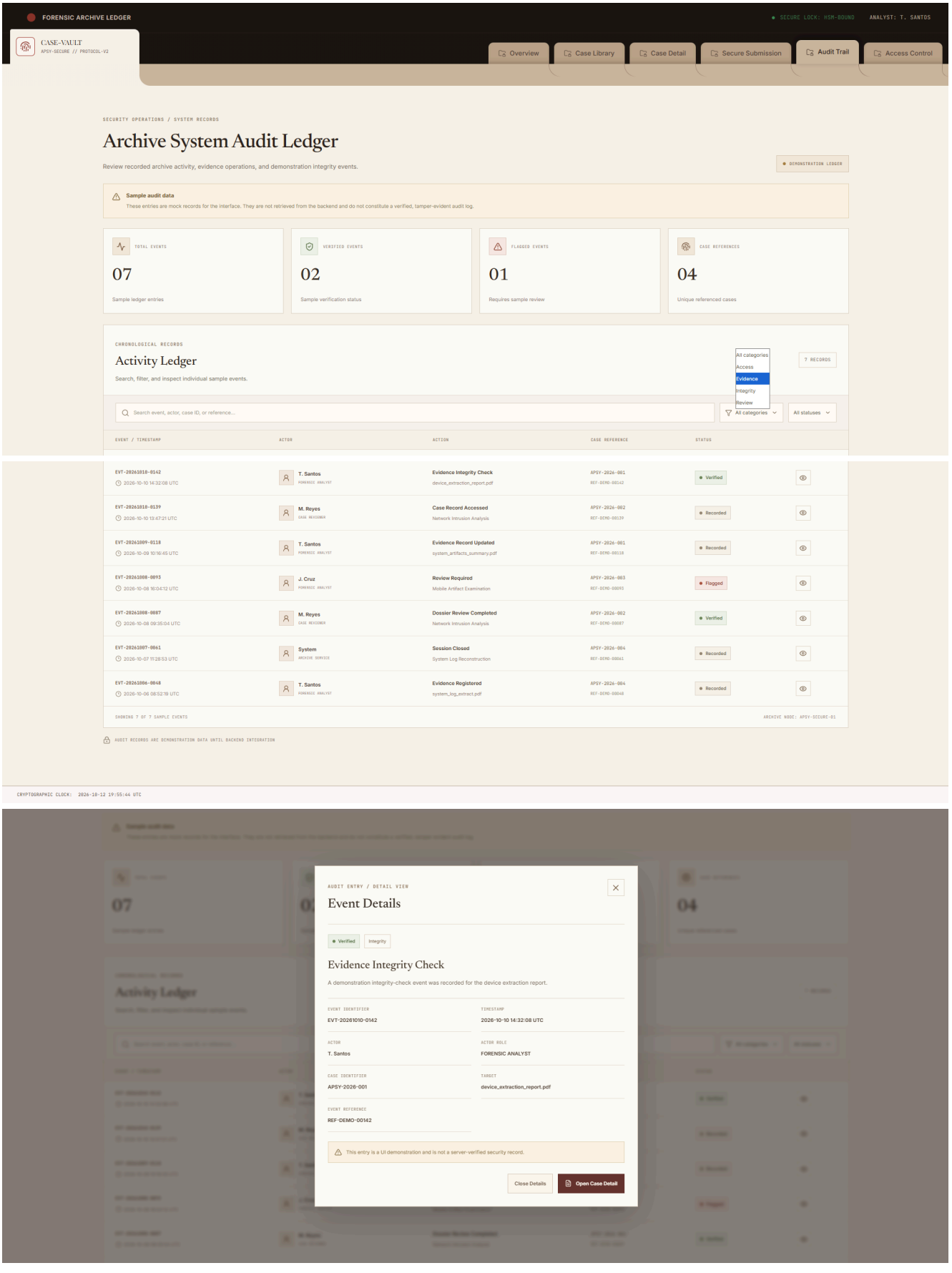
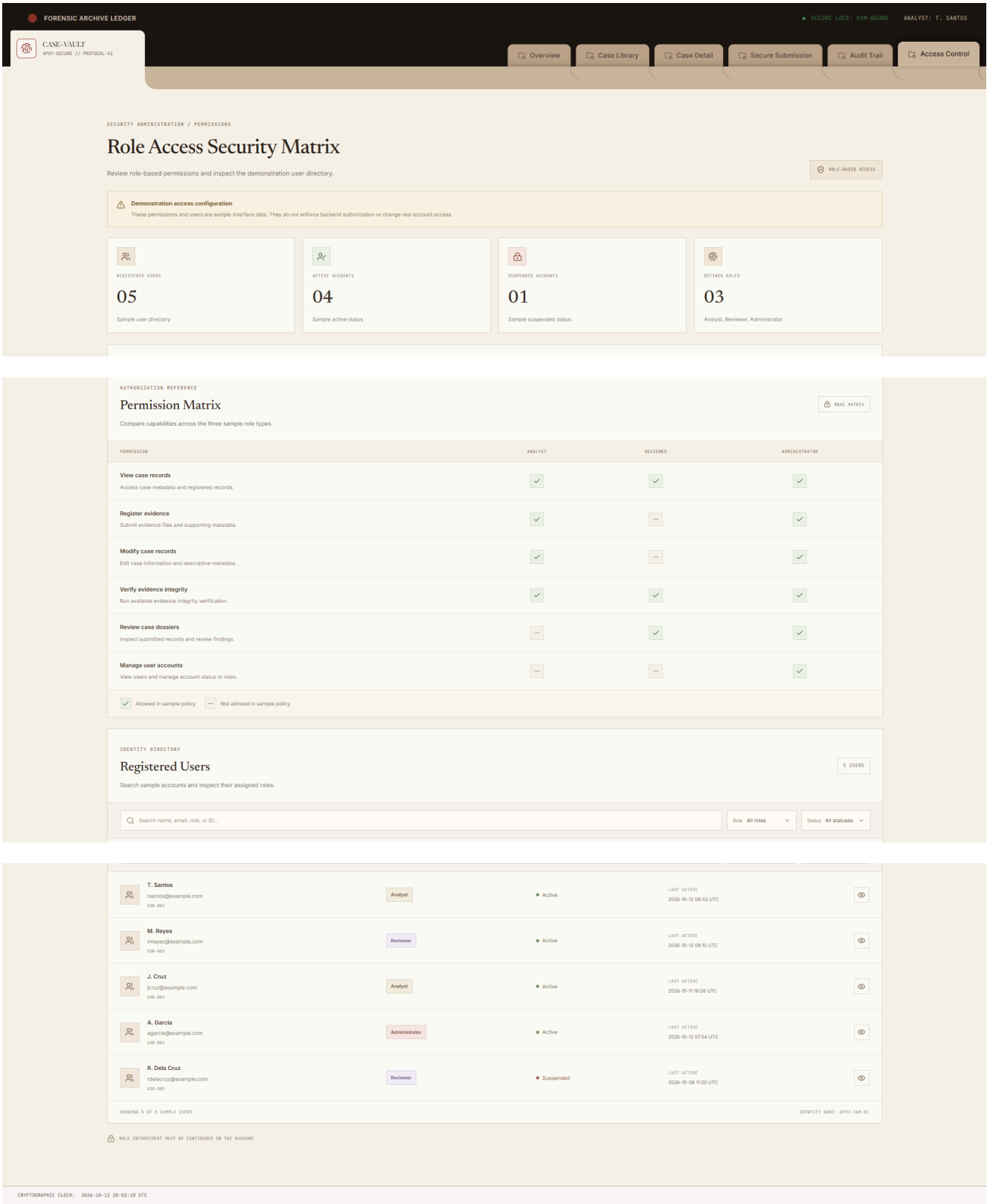

FORENSIC ARCHIVE LEDGER
SECURE LOCK: HSM-BOUND
ANALYST: T. SANTOS
CASE-VAULT
APSY-SECURE // PROTOCOL-V2
Overview
Case Library
Case Detail
Secure Submission
Audit Trail
Access Control
SECURITY OPERATIONS / SYSTEM RECORDS
Archive System Audit Ledger
Review recorded archive activity, evidence operations, and demonstration integrity events.
DEMONSTRATION LEDGER
Sample audit data
These entries are mock records for the interface. They are not retrieved from the backend and do not constitute a verified, tamper-evident audit log.
TOTAL EVENTS
07
Sample ledger entries
VERIFIED EVENTS
02
Sample verification status
FLAGGED EVENTS
01
Requires sample review
CASE REFERENCES
04
Unique referenced cases
CHRONOLOGICAL RECORDS
Activity Ledger
Search, filter, and inspect individual sample events.
All categories
Access
Evidence
Integrity
Review
7 RECORDS
Search event, actor, case ID, or reference...
All categories
All statuses
EVENT / TIMESTAMP
ACTOR
ACTION
CASE REFERENCE
STATUS
EVT-20261010-0142
2026-10-10 14:32:08 UTC
T. Santos
FORENSIC ANALYST
Evidence Integrity Check
device_extraction_report.pdf
APSY-2026-001
REF-DEMO-00142
Verified
EVT-20261010-0139
2026-10-10 13:47:21 UTC
M. Reyes
CASE REVIEWER
Case Record Accessed
Network Intrusion Analysis
APSY-2026-002
REF-DEMO-00139
Recorded
EVT-20261009-0118
2026-10-09 10:16:45 UTC
T. Santos
FORENSIC ANALYST
Evidence Record Updated
system_artifacts_summary.pdf
APSY-2026-001
REF-DEMO-00118
Recorded
EVT-20261008-0093
2026-10-08 16:04:12 UTC
J. Cruz
FORENSIC ANALYST
Review Required
Mobile Artifact Examination
APSY-2026-003
REF-DEMO-00093
Flagged
EVT-20261008-0087
2026-10-08 09:35:04 UTC
M. Reyes
CASE REVIEWER
Dossier Review Completed
Network Intrusion Analysis
APSY-2026-002
REF-DEMO-00087
Verified
EVT-20261007-0061
2026-10-07 11:28:53 UTC
System
ARCHIVE SERVICE
Session Closed
System Log Reconstruction
APSY-2026-004
REF-DEMO-00061
Recorded
EVT-20261006-0048
2026-10-06 08:52:19 UTC
T. Santos
FORENSIC ANALYST
Evidence Registered
system_log_extract.pdf
APSY-2026-004
REF-DEMO-00048
Recorded
SHOWING 7 OF 7 SAMPLE EVENTS
ARCHIVE NODE: APSY-SECURE-01
AUDIT RECORDS ARE DEMONSTRATION DATA UNTIL BACKEND INTEGRATION
CRYPTOGRAPHIC CLOCK: 2026-10-12 19:55:44 UTC
AUDIT ENTRY / DETAIL VIEW
Event Details
Verified
Integrity
Evidence Integrity Check
A demonstration integrity-check event was recorded for the device extraction report.
EVENT IDENTIFIER
EVT-20261010-0142
TIMESTAMP
2026-10-10 14:32:08 UTC
ACTOR
T. Santos
ACTOR ROLE
FORENSIC ANALYST
CASE IDENTIFIER
APSY-2026-001
TARGET
device_extraction_report.pdf
EVENT REFERENCE
REF-DEMO-00142
This entry is a UI demonstration and is not a server-verified security record.
Close Details
Open Case Detail

Figure 9.2.5. The Audit Trail page presents system activity records to support the monitoring and review of actions performed within Case Vault. It is intended to help users trace recorded activities and examine the integrity and history of forensic records.



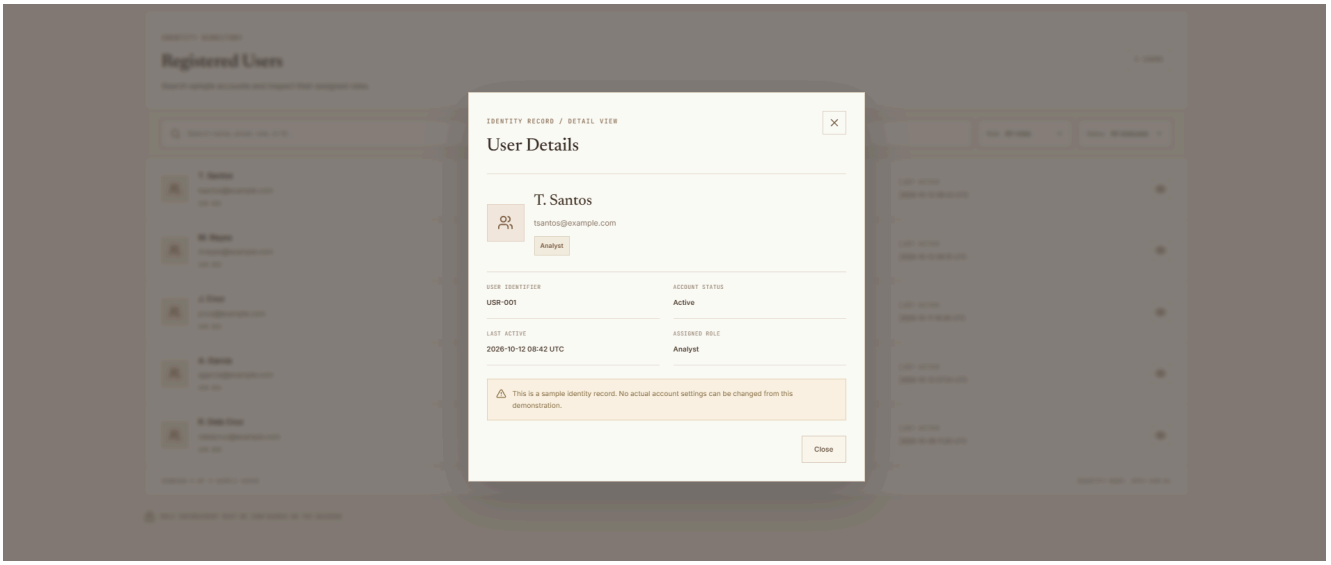


Figure 9.2.6. The Access Control page displays registered user accounts, assigned roles, account statuses, and permission levels. It provides an overview of user access privileges to support the management and monitoring of access to system resources.

9.3 Transaction Page

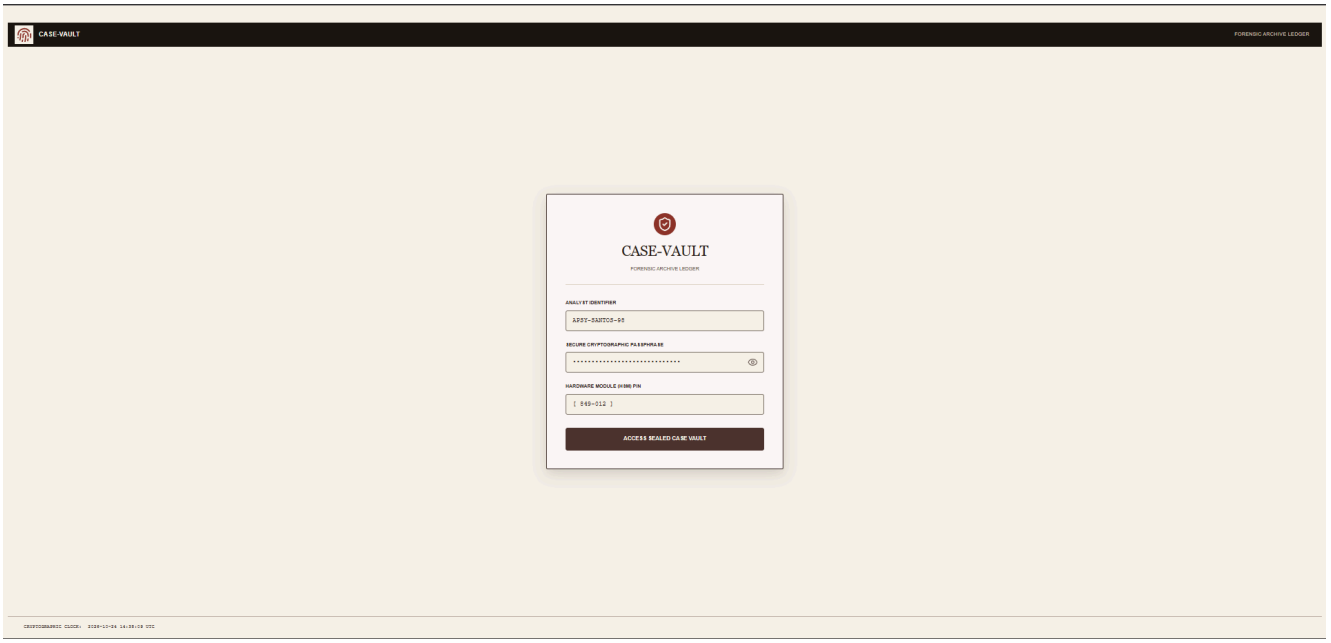


Figure 9.3.1. The transaction page documents the login transaction, displaying relevant authentication details when an authorized user accesses the Case Vault system.



Add New Cases by toggling the +NewCase Button at the upper right

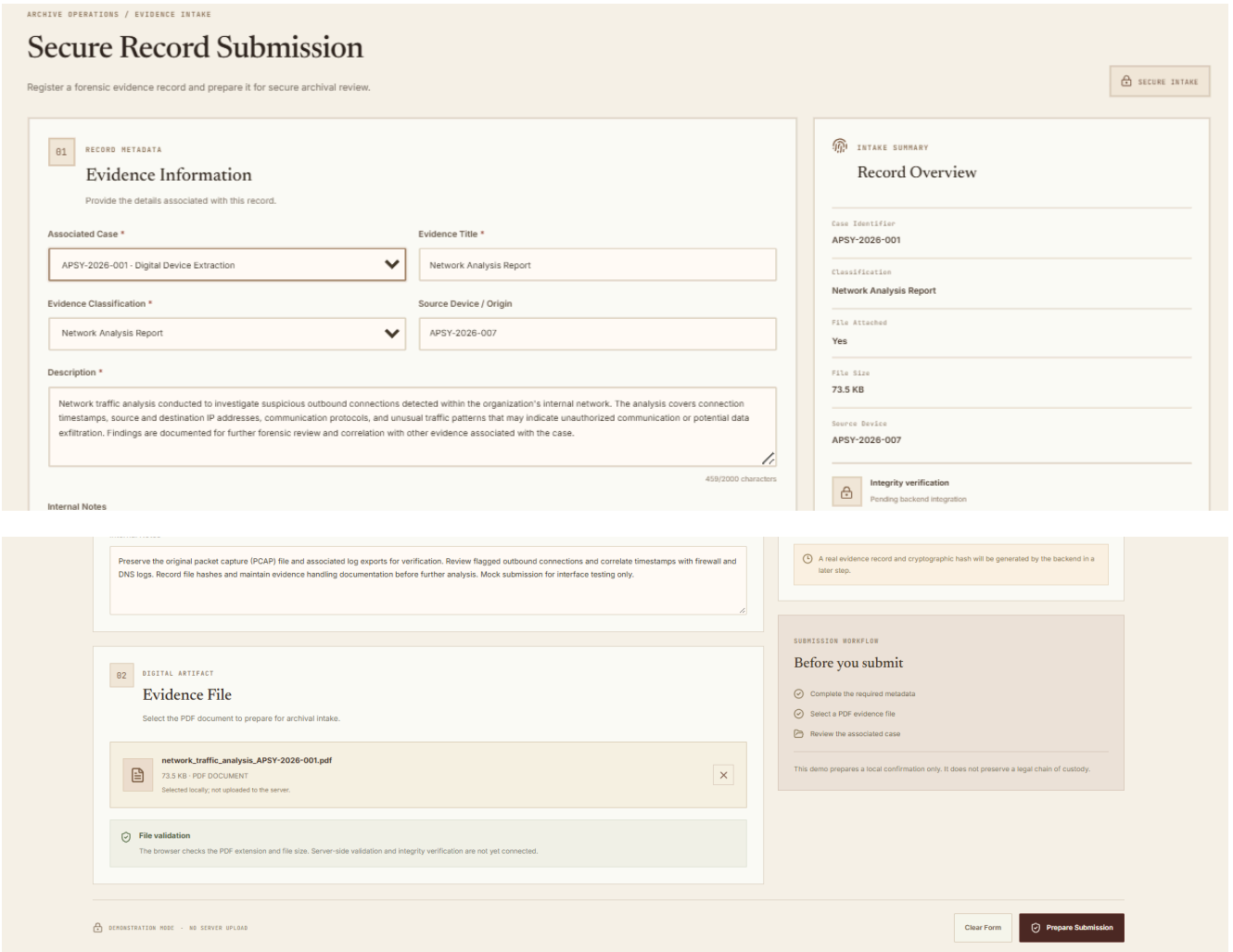


Figure 9.3.2.The Secure Submission page provides an interface for entering forensic case information and preparing supporting documentation for submission. It organizes the information associated with a case and represents the system's primary case-entry workflow.

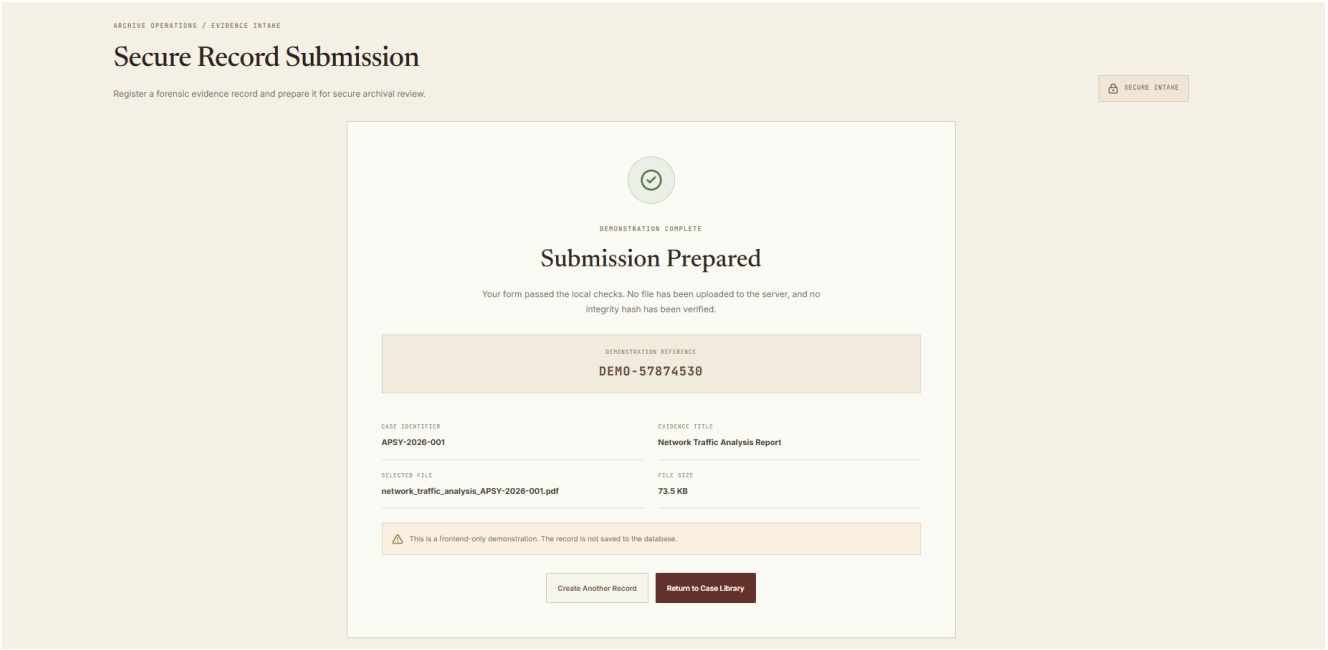


Figure 9.3.3. The document-selection or submission state illustrates the user's interaction with the evidence submission form, including the preparation of supporting documentation before completing the process. This interface helps demonstrate the intended workflow for associating documents with a forensic case.

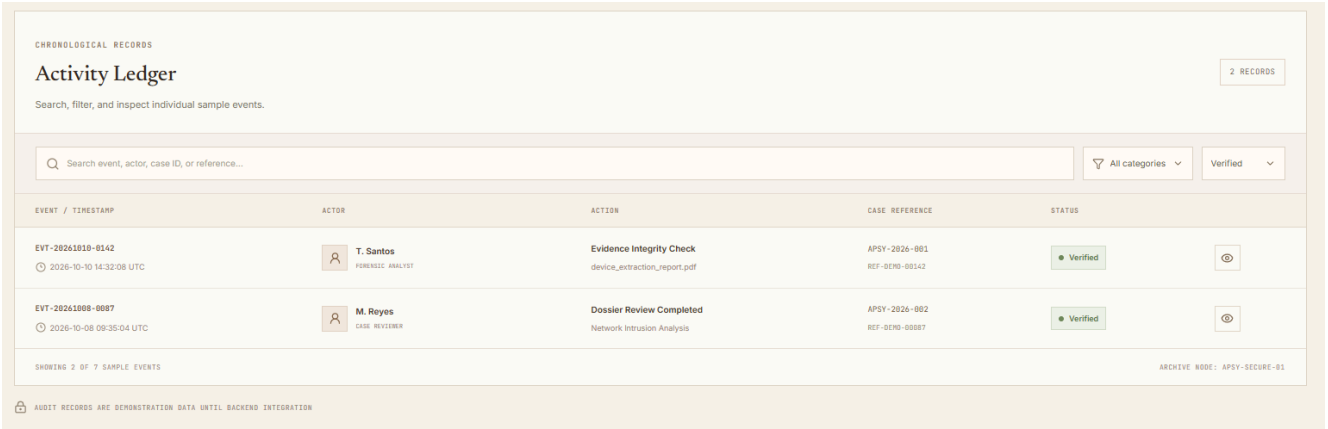


Figure 9.3.4. The Activity Ledger Filtering and Searching System allows users to locate and review recorded activities using keywords and available filters, making it easier to trace system operations for monitoring and auditing purposes.



Figure 9.3.5. The Registered User Filtering and Searching System allows users to locate specific user accounts using keywords and available filters. It simplifies the retrieval and review of registered user information for efficient account management and monitoring.

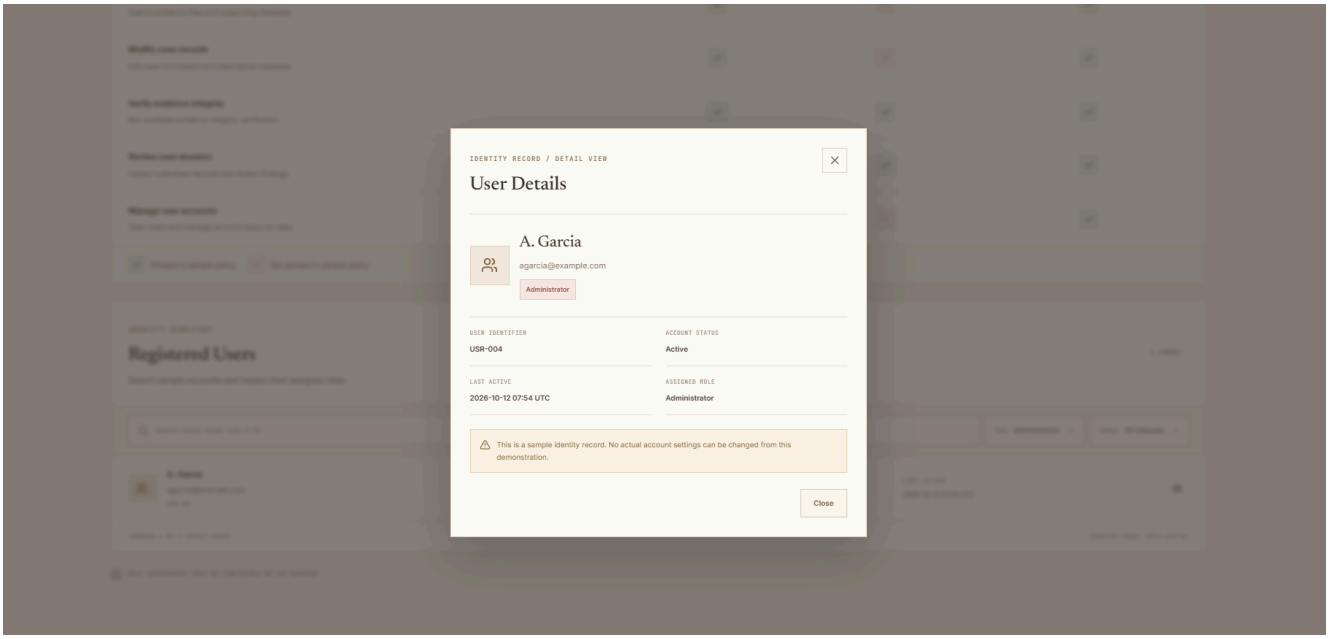


Figure 9.3.6. The User Details interface allows users to view detailed information about a selected registered account, providing a clearer overview of user profiles and account-related information for monitoring and management purposes.

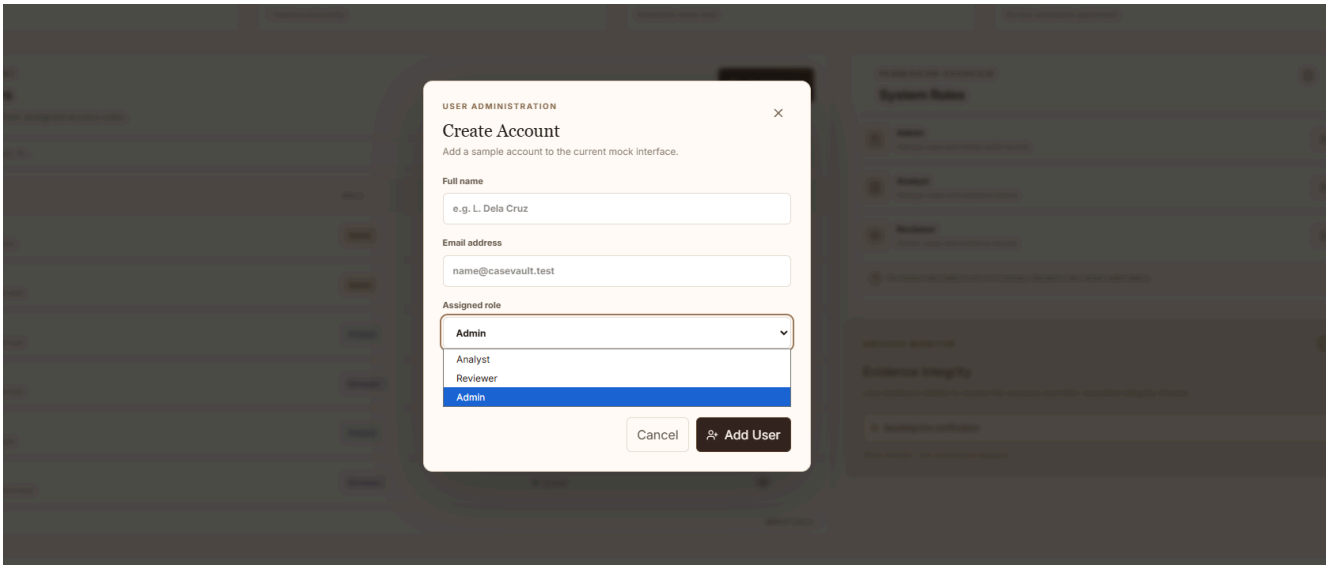


Figure 9.3.7. The Add User feature allows administrators to register new user accounts by entering the required account information and assigning an appropriate role, supporting organized user access management within the system.

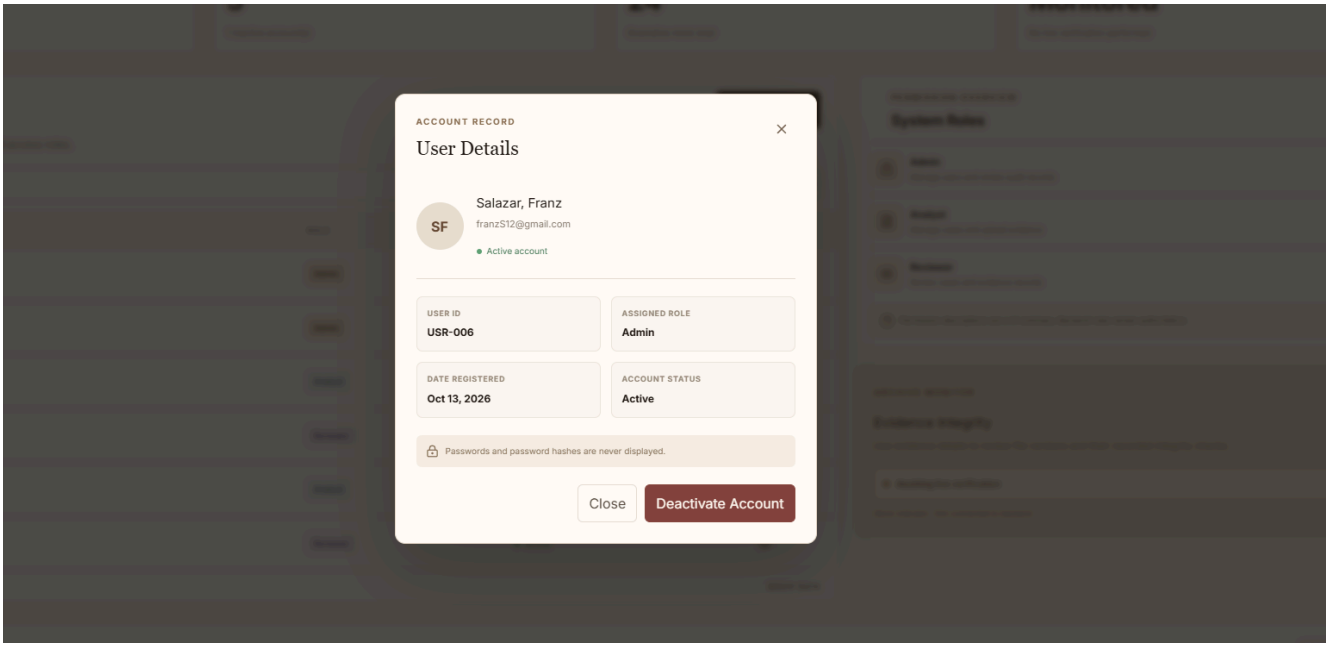


Figure 9.3.8. The Deactivate Account feature allows administrators to disable selected user accounts when access is no longer needed. This helps manage active users and maintain control over system access.

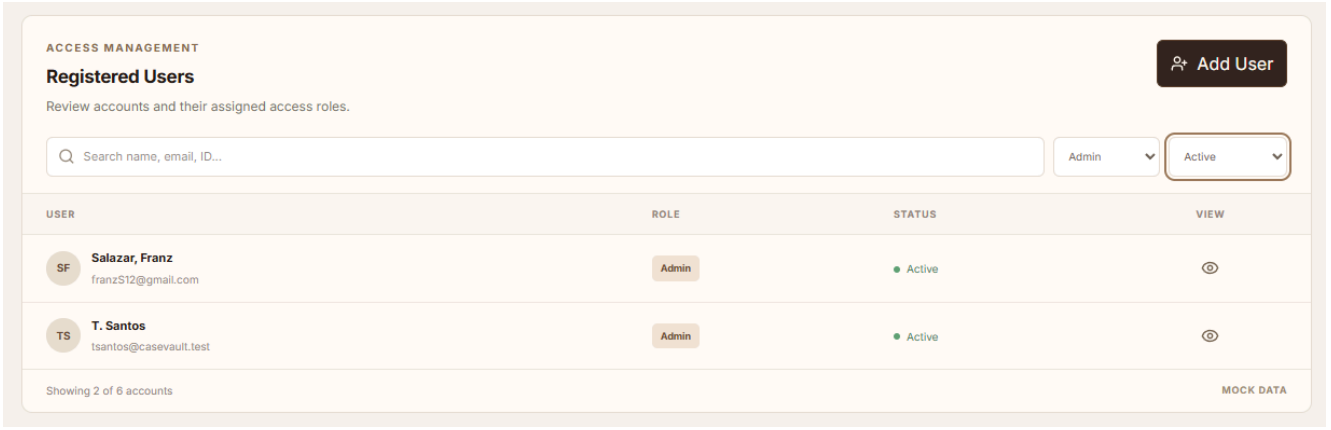


Figure 9.3.9. The filtering and searching function for Registered Users allows administrators to quickly locate specific user accounts by entering keywords or selecting available filters, such as user role and account status. This simplifies user record retrieval and supports efficient account monitoring and management.

9.4 Other Major Interfaces

Admin Panel

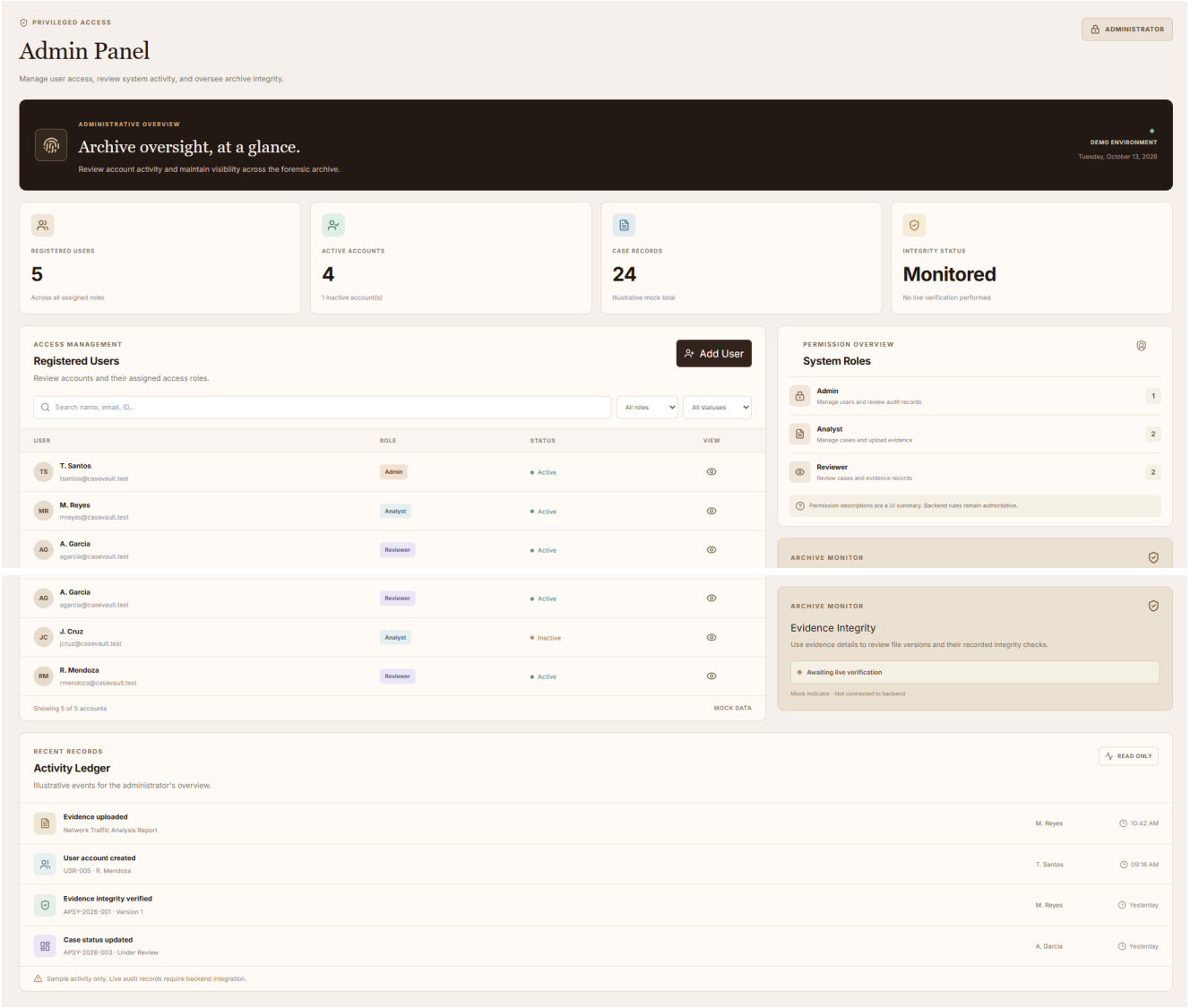


Figure 9.4.1 The Admin Panel provides a centralized interface for managing registered users, monitoring account status, reviewing system activity, and viewing system integrity indicators. It allows administrators to search and filter user records, view account details, and manage account activation status. The panel also presents an overview of user roles and recent activity to support efficient system administration and monitoring.

10. SYSTEM IMPLEMENTATION

The supplied system was developed as a Python FastAPI backend. Application startup initializes MongoDB and registers Beanie document models. Routers handle HTTP requests; Pydantic schemas validate request and response data; authentication dependencies validate bearer tokens and permissions; services provide account creation, audit logging, file storage/integrity checks, and database seeding. MongoDB stores application metadata and GridFS stores uploaded file bytes. The source repository does not contain the proposed Next.js frontend.

10.1 Development Tools and Technologies

Tool/Technology	Purpose
Python 3.12	Backend programming language, as specified in the README.
FastAPI	Asynchronous web API framework and OpenAPI/Swagger documentation.
Pydantic and pydantic-settings	Request/response validation and environment-based application settings.
MongoDB Atlas	Database deployment described in the README; a replica set is required for multi-document transactions.
Motor	Asynchronous MongoDB driver.
Beanie	Object-document mapping and MongoDB document models.
MongoDB GridFS	Storage and retrieval of uploaded file contents.
Passlib and bcrypt	Password hashing and password verification.
python-jose	HS256 JWT signing and token decoding.
hashlib	SHA-256 calculation for evidence upload and integrity verification.
certifi	CA certificate bundle used for TLS database connection.
Uvicorn	ASGI server used to run the FastAPI application.
Visual Studio Code / PyCharm	Listed as proposed development environments in the proposal; actual IDE use cannot be verified from source.

Next.js, JavaScript/TypeScript, Tailwind CSS	Proposed frontend technologies; frontend code was not supplied.
Git / GitHub	Version control and hosting described in the proposal/README; repository hosting details were not supplied.

10.2 Major System Features

1. **Authentication:** Email/password login, bcrypt password verification, signed bearer JWTs with an eight-hour expiry, current-user lookup, inactive-account rejection, and authenticated password change.
2. **Role-based authorization:** Admin, analyst, and reviewer roles are seeded with permission records. Each protected endpoint declares the permission it requires.
3. **User administration:** Administrators can create users, list users, change a user's role or active status, and list roles. New user creation and its audit record use a MongoDB transaction.
4. **Case management:** Authorized users can create cases with unique case numbers, list/filter cases, retrieve a case, update case status, and soft-delete a case where permitted.
5. **PDF evidence management:** Authorized users can upload PDF evidence to an active case, list evidence by case, list versions, and download version files.
6. **Version history and integrity verification:** Replacement uploads add new version records without overwriting older version records. Each version has a SHA-256 hash and can be checked on demand; a mismatch sets the tamper flag and writes an audit event.
7. **Tags:** Authorized users can create/list tags and assign/list/remove tags on cases. Duplicate tag names and duplicate case-tag assignments are rejected.
8. **Audit records:** Selected events are stored with actor, action, time, and optional case/evidence context. Administrators can filter audit results.
9. **Health and API documentation:** /health reports the API status, and FastAPI's /docs provides interactive API documentation under the default setup.

11. TRANSACTION PROCESSING

Transaction 1: User Login

Input:

- Email
- Password
- Sent as a JSON body. No token is needed because this is how the token is obtained.

Validation:

- **email** must be a correctly formatted email address and password must be provided (otherwise 422 validation error).
- An account with that email must exist and the password must match the stored bcrypt hash; otherwise **401 "Invalid email or password".** The same message is used for both cases so an attacker cannot tell which emails are registered.
- The account must be active (is_active = true); otherwise **403 "This account has been deactivated".**

Processing:

1. The system receives the credentials and validates their format.
2. It looks up the user by email in the users collection.
3. It compares the typed password with the stored hash using bcrypt (the password is never decrypted).
4. It creates a signed JWT access token containing the user id and role id, valid for 8 hours.
5. It records the login in the audit log.

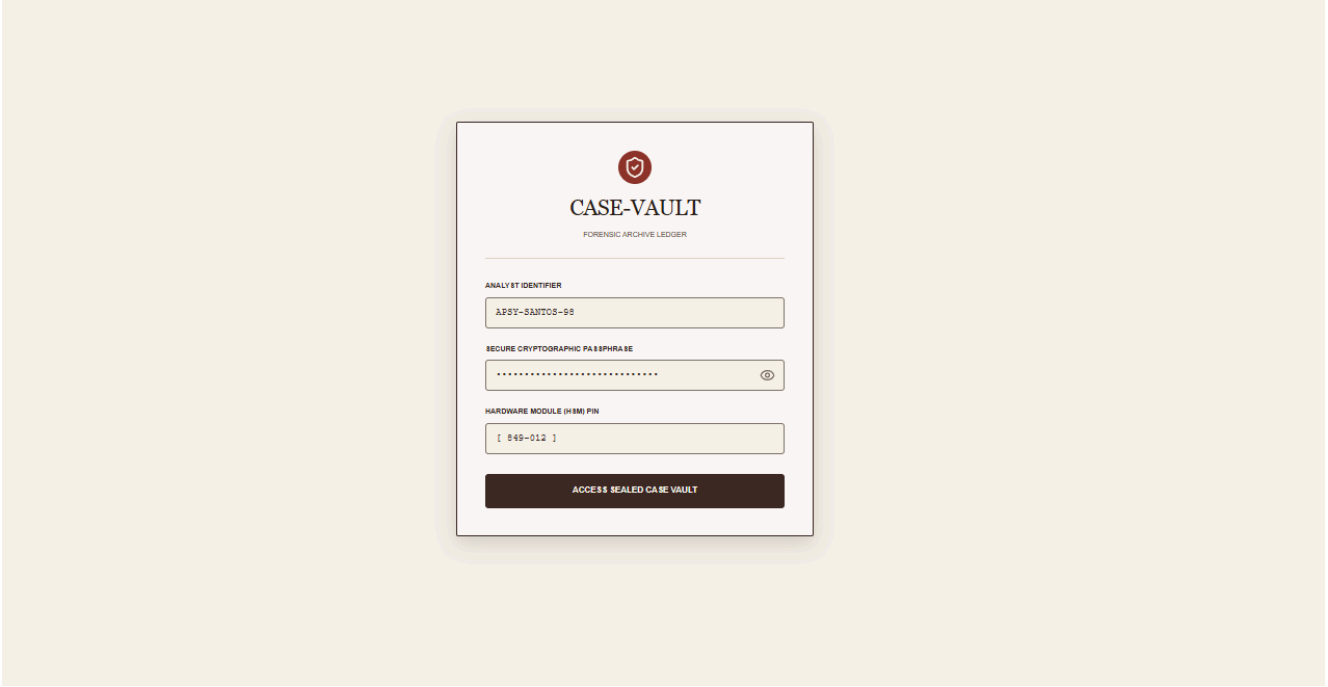
Database Update:

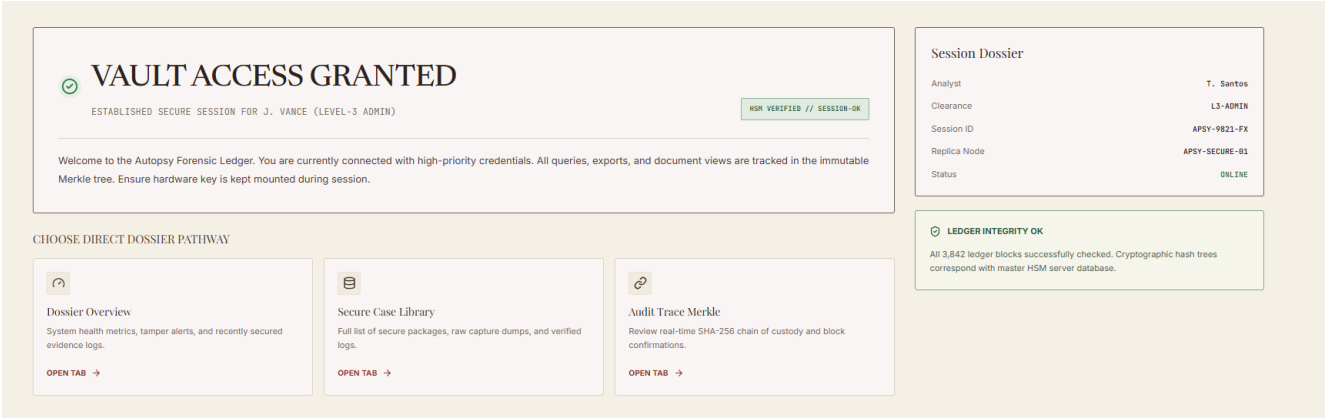
- audit_transactions: +1 record with action user_logged_in and the user as the actor.
- No change to the users collection.

Confirmation/Output:

- **200 OK** with access_token and token_type ("bearer").
- The token is sent as Authorization: Bearer <token> on every later request.
- Failure responses: 401 (wrong email or password), 403 (deactivated account), 422 (invalid input).

Screenshot:





Transaction 2: Create User

Input:

- name, email, password, role_id (JSON body).
- The admin's access token in the Authorization header.

Validation:

- The caller must be logged in with a valid token and an active account (401 / 403).
- The caller must have the user:manage permission, which only the admin role has (403 for analyst and reviewer).
- name must be 2 to 150 characters, email must be a valid format, password must be 8 to 128 characters (422).
- role_id must belong to an existing role; otherwise **404 "Role not found".**

- The email must not already be registered; otherwise **409 "A user with this email already exists".**

Processing:

1. The system verifies the token and the user:manage permission.
2. It validates the request body and checks for a duplicate email and a valid role.
3. It hashes the password with bcrypt so the plain password is never stored.
4. It opens a MongoDB transaction, inserts the new user and the audit entry inside it, then commits.

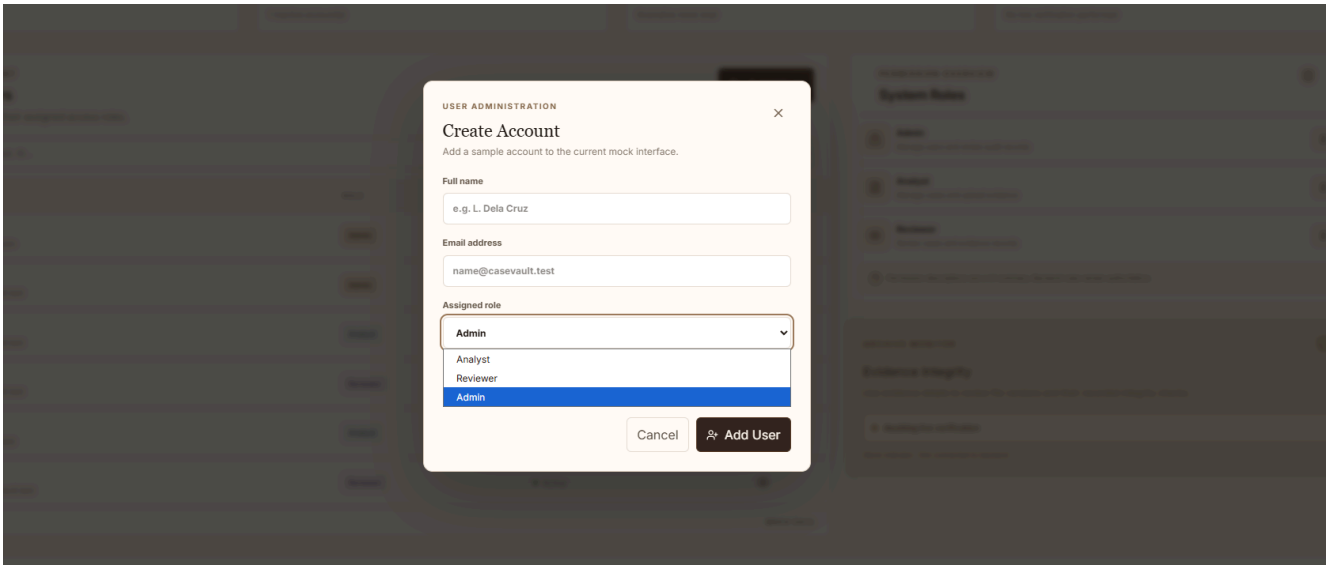
Database Update:

- users: +1 document (role_id, name, email, hashed_password, is_active = true, created_at, updated_at).
- audit_transactions: +1 document with action user_created and metadata new_user_id and role_id.
- Both writes happen in one transaction, so either both are saved or neither is.

Confirmation/Output:

- **201 Created** with the new account (id, name, email, role_id, is_active, created_at, updated_at).
- The password and its hash are never returned.

Screenshot:



Transaction 3: Create Case

Input:

- case_number (for example CV-2026-0001)

- case_type (for example Digital Forensics)
- The user's access token. The creator is taken from the token, not typed in.

Validation:

- The user must be logged in with an active account (401 / 403).
- The user must have a case:create; a reviewer receives 403.
- case_number must be 2 to 50 characters and case_type 2 to 100 characters (422).
- case_number must be unique; otherwise 409 "A case with this number already exists".

Processing:

1. The system verifies the token and permission.
2. It validates the body and checks that no case already uses the case number.
3. It builds the case with status open, created_by set to the logged-in user, submission_date set to now and no deletion date.
4. It opens a MongoDB transaction, inserts the case and the audit entry, then commits.

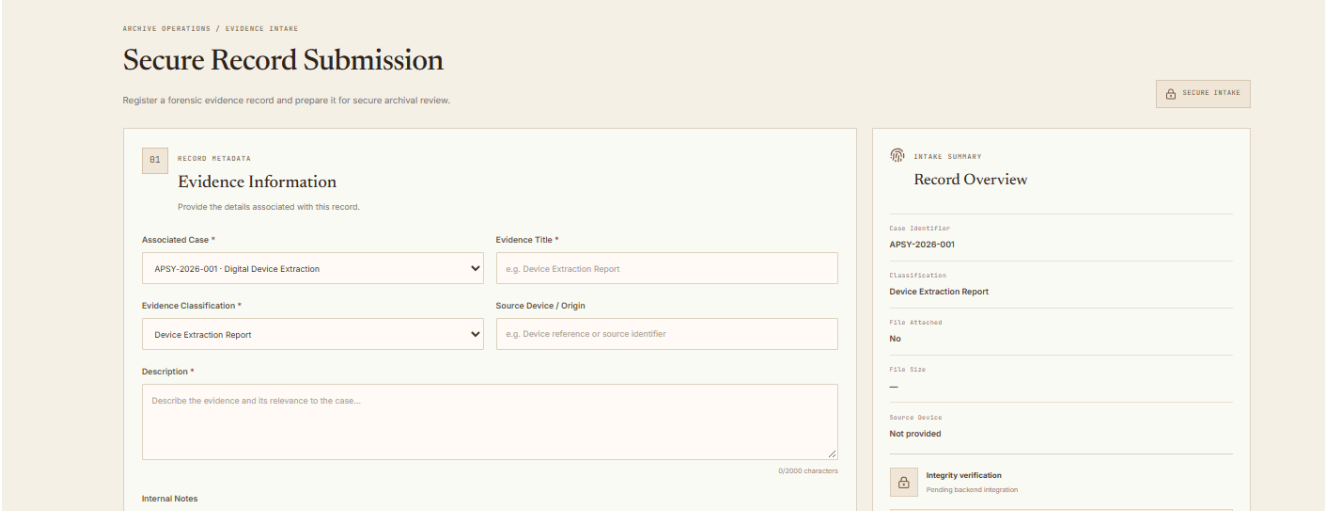
Database Update:

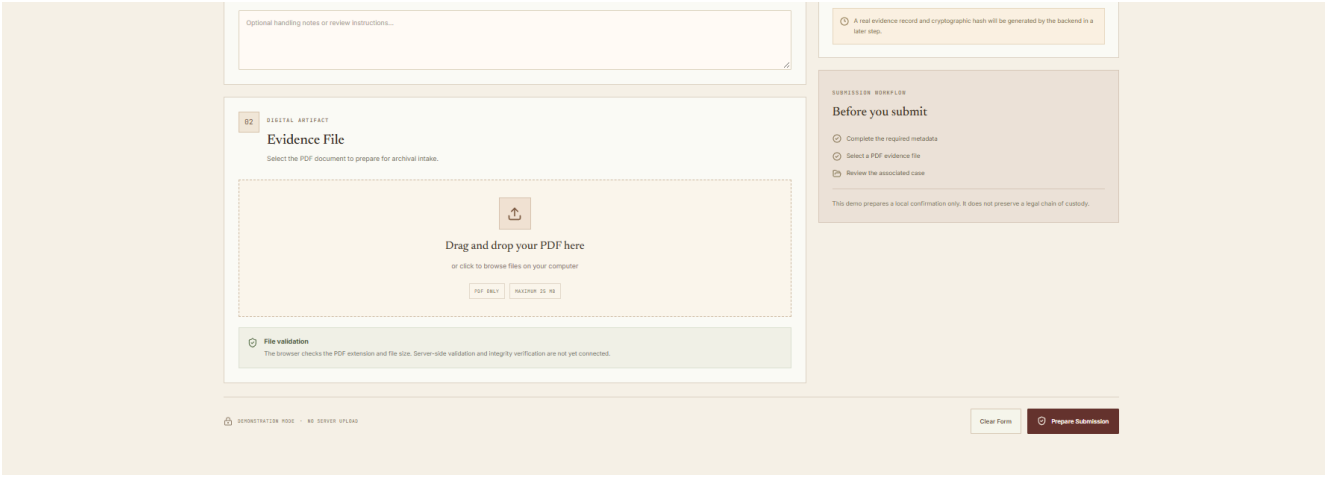
- cases: +1 document (case_number, case_type, status = open, created_by, submission_date, deleted_at = empty, created_at, updated_at).
- audit_transactions: +1 document with action case_created and the new case_id.
- Both writes are atomic (one transaction).

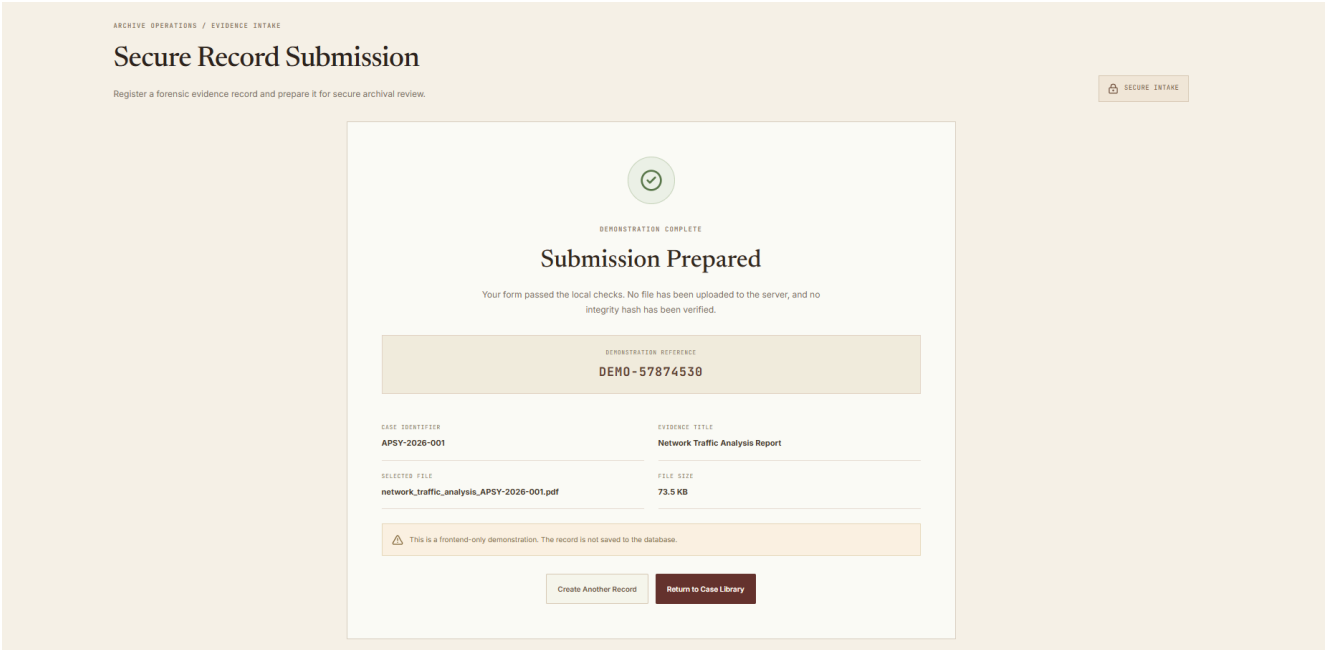
- **Confirmation/Output:**

  201 Created with the case (id, case_number, case_type, status, created_by, submission_date, created_at, updated_at).

Screenshot:







Transaction 4: View and Search Cases

Input:

- Optional query filters on the list: status, case_type, q (partial case number).
- Or a case_id in the path to view one case.

Validation:

- The user must be logged in and have case:view (all three roles do).
- Filters are optional. Soft-deleted cases are never returned.
- Viewing one case: an unknown or deleted id returns 404 "Case not found".

Processing:

1. The system verifies the token and permission.

2. It builds a filter that starts with "not deleted" and adds each filter that was supplied: exact status, exact case_type, and q as a case-insensitive partial match on the case number.
3. It runs the query and returns the matching cases.

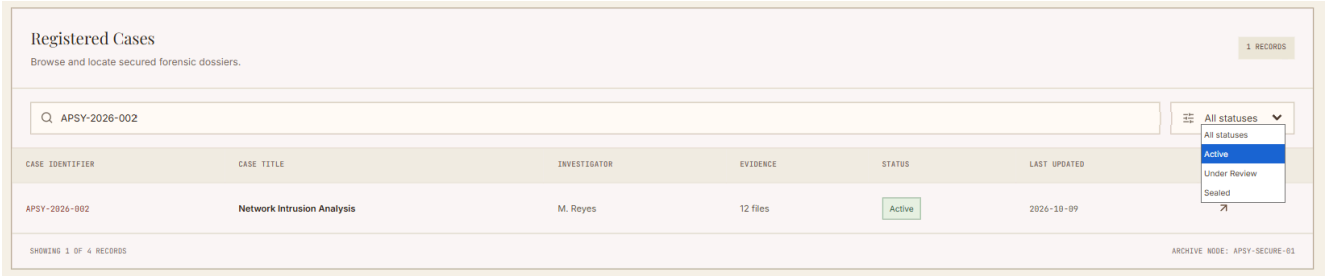
Database Update:

- None. This is a read-only transaction.

Confirmation/Output:

- 200 OK with a list of cases (an empty list if nothing matches), or a single case for GET /cases/{case_id}.

Screenshot:



Transaction 5: Update Case Status

Input:x

- case_id (path)
- status (JSON body, for example under_review)

Validation:

- The user must be logged in and have case:update_status; an analyst receives 403.
- The case must exist and must not be soft-deleted; otherwise 404 "Case not found".
- status is free text (no fixed list yet). If it is omitted nothing changes.

Processing:

1. The system verifies the token and permission and loads the case.
2. It sets the new status and refreshes updated_at, then saves the case.
3. It writes an audit entry recording the new status.

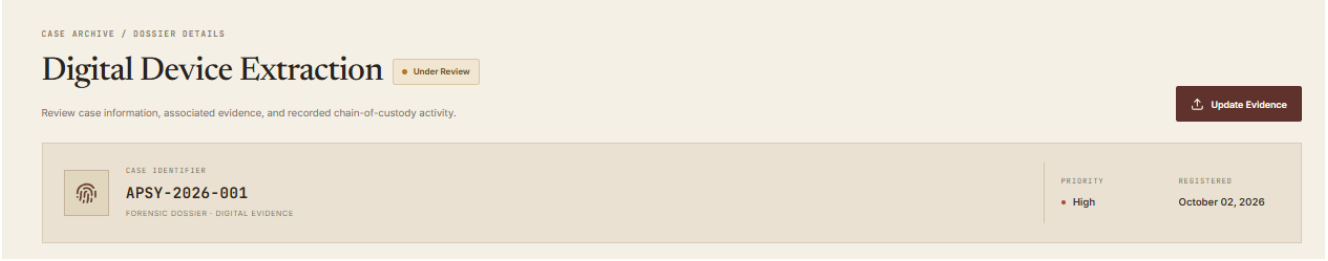
Database Update:

- cases: the existing document is updated (status, updated_at). No new document is created.
- audit_transactions: +1 document with action case_status_updated and metadata new_status.

Confirmation/Output:

- 200 OK with the updated case.

Screenshot:



Transaction 6: Create Tag

Input:

- name (JSON body, for example urgent)

Validation:

- The user must be logged in and have case:create; a reviewer receives 403.
- name must be 1 to 50 characters (422).
- The tag name must be unique; otherwise 409 "This tag already exists".

Processing:

1. The system verifies the token and permission.
2. It checks that no tag with the same name exists.
3. It creates the tag.

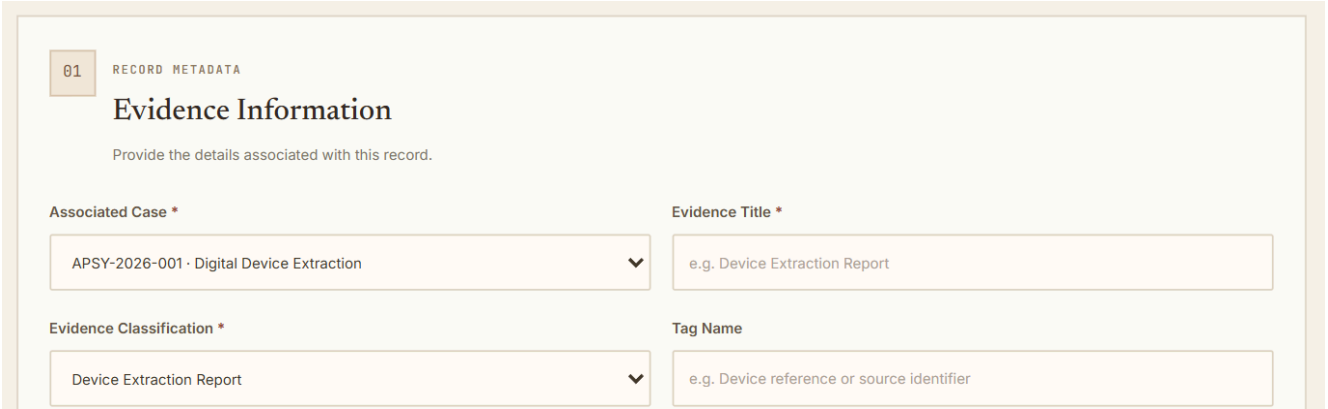
Database Update:

- tags: +1 document (name).

Confirmation/Output:

- 201 Created with the tag (id, name).
- GET /tags lists all tags for any user with case:view.

Screenshot:



Transaction 7: Assign Tag to Case

Input:

- case_id (path)
- tag_id (JSON body)

Validation:

- The user must be logged in and have case:update_status.
- The case must exist and not be soft-deleted (404 "Case not found").
- The tag must exist (404 "Tag not found").
- The tag must not already be assigned to that case; otherwise 409 "This tag is already assigned to this case".

Processing:

1. The system verifies the token and permission and loads the case and the tag.
2. It checks that the case and tag are not already linked.
3. It creates the link, recording who assigned the tag and when.
4. It writes an audit entry.

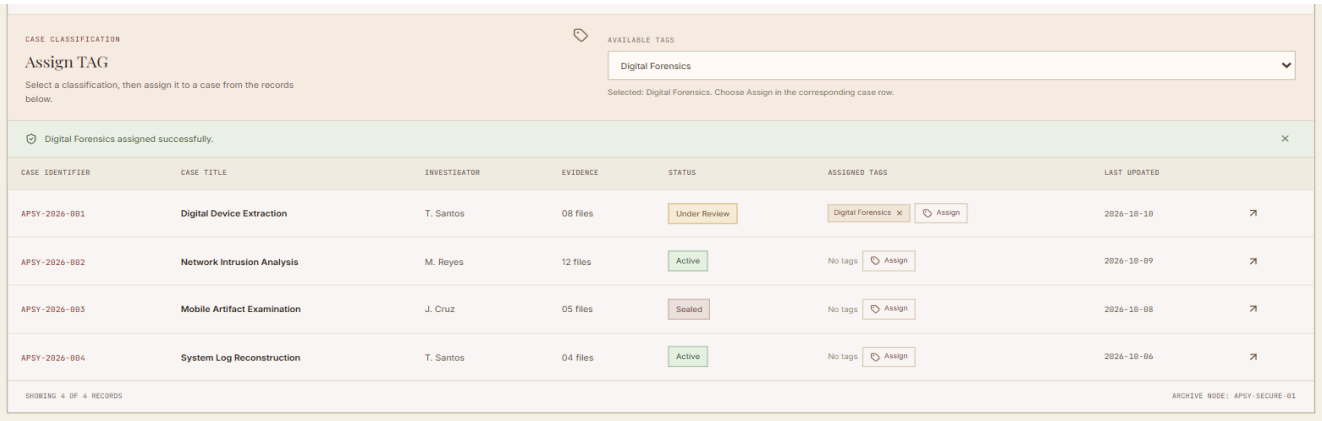
Database Update:

- case_tags: +1 document (case_id, tag_id, assigned_by, assigned_at).
- audit_transactions: +1 document with action tag_assigned and metadata tag_id and tag_name.

Confirmation/Output:

- 201 Created with the link record.
- GET /cases/{case_id}/tags returns the tags currently on the case.

Screenshot:



Transaction 8: Remove Tag from Case

Input:

- case_id (path)
- tag_id (path)

Validation:

- The user must be logged in and have case:update_status.
- The tag must currently be assigned to that case; otherwise 404 "This tag is not assigned to this case".

Processing:

1. The system verifies the token and permission.
2. It finds the link between the case and the tag.
3. It removes only that link. The tag itself stays available for other cases.
4. It writes an audit entry.

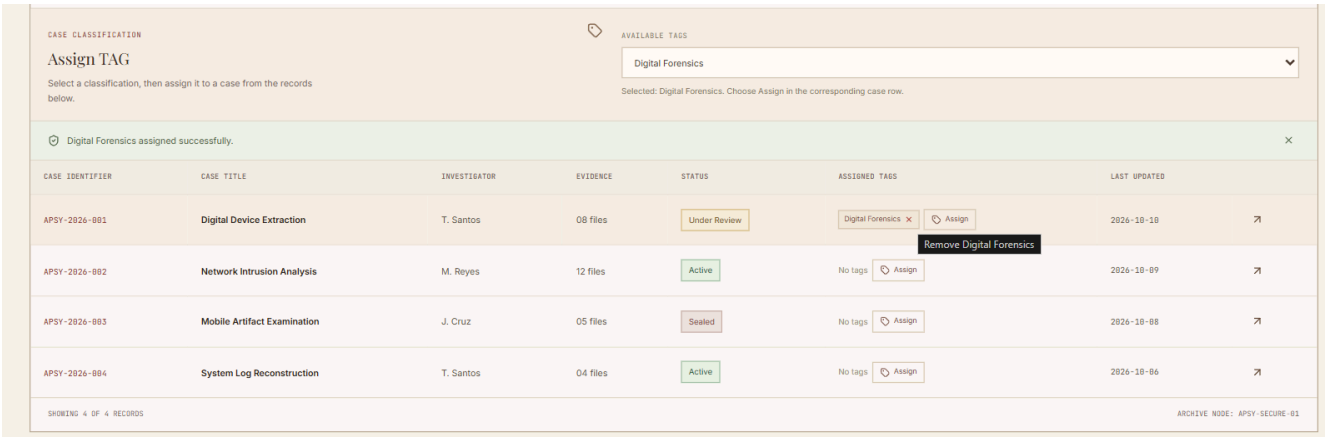
Database Update:

- case_tags: the link document is removed (-1).
- tags: unchanged.
- audit_transactions: +1 document with action tag_removed and metadata tag_id.

Confirmation/Output:

- 204 No Content.
- GET /cases/{case_id}/tags no longer lists the tag.

Screenshot:



Transaction 9: Upload Evidence (First Version)

Input:

- Sent as a form with a file (multipart/form-data):

- case_id, evidence_type, title
- file (a PDF)

Validation:

- The user must be logged in and have evidence:upload; a reviewer receives 403.
- The file must be a PDF; otherwise 400 "Only PDF files are accepted".
- The case must exist and must not be soft-deleted; otherwise 404 "Case not found".
- All fields are required. evidence_type is 2 to 50 characters and title is 2 to 255 characters. (422).

Processing:

1. The system verifies the token and permission, the file type and the case.
2. It reads the file and computes its SHA-256 hash (the file's fingerprint).
3. It stores the file in GridFS, which returns a file id.
4. It builds the evidence record and its first version (version 1) holding the hash and the GridFS id.
5. It opens a MongoDB transaction, inserts the evidence and the version, points the evidence to its current version, writes the audit entry, then commits.

Database Update:

- evidence: +1 document (case_id, submitted_by, evidence_type, title, current_version_id, created_at).
- evidence_versions: +1 document (version_no = 1, file_name, file_type, file_hash, storage_path, is_tamper_flagged = false, uploaded_at).
- GridFS (fs.files and fs.chunks): the PDF itself is stored.
- audit_transactions: +1 document with action evidence_uploaded, metadata version_no and file_hash.
- The three records are written atomically. The file is stored just before the transaction starts.

Confirmation/Output:

- 201 Created with the evidence record (id, case_id, submitted_by, evidence_type, title, current_version_id, created_at).

Screenshot:

Transaction 10: Upload New Evidence Version

Input:

- evidence_id (path)
- file (a replacement PDF, sent as a form file)

Validation:

- The user must be logged in and have evidence:upload.
- The evidence must exist; otherwise 404 "Evidence not found".
- The file must be a PDF; otherwise 400 "Only PDF files are accepted".

Processing:

1. The system verifies the token and permission and loads the evidence.
2. It counts the existing versions to work out the next version number.
3. It hashes the new file and stores it in GridFS. Earlier files are not touched.
4. It opens a MongoDB transaction, inserts the new version, moves the evidence's current-version pointer to it, writes the audit entry, then commits.

Database Update:

- evidence_versions: +1 document (version_no = previous + 1, new file_hash, new storage_path).
- evidence: current_version_id is updated to the new version.
- GridFS: the new file is stored.
- audit_transactions: +1 document with action evidence_version_added.
- Old versions are never changed or deleted, which keeps the full history immutable.

Confirmation/Output:

- 201 Created with the version (id, evidence_id, uploaded_by, version_no, file_name, file_type, file_hash, is_tamper_flagged, uploaded_at).

Transaction 11: View Evidence and Version History

Input:

- case_id (path) to list the evidence under a case.
- Or evidence_id (path) to list the versions of one evidence record.

Validation:

- The user must be logged in and have evidence:view (all three roles do).
- If nothing matches, an empty list is returned.

Processing:

1. The system verifies the token and permission.
2. It looks up the evidence by case, or the versions by evidence.
3. It returns the records.

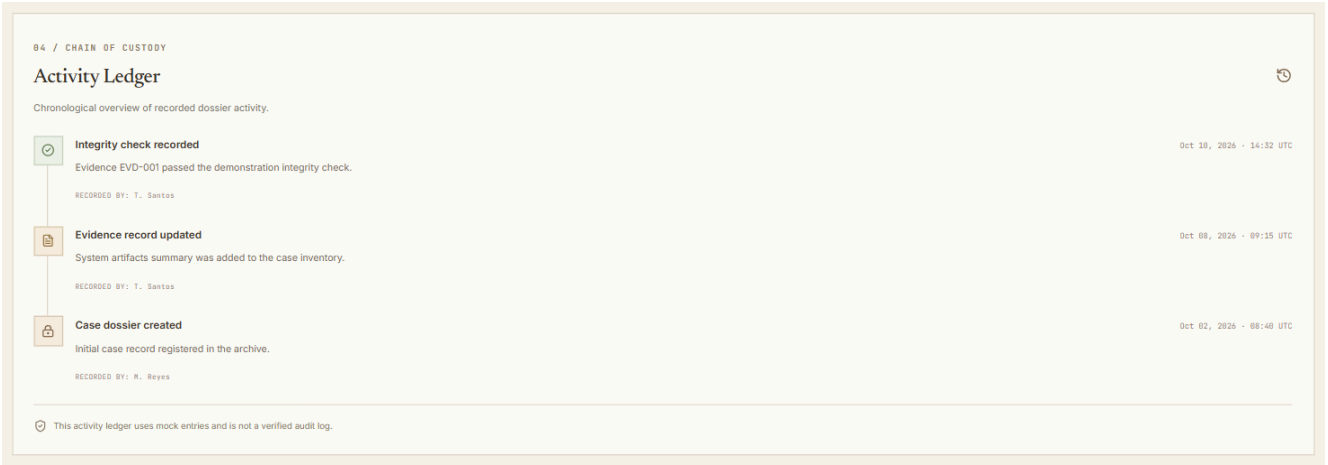
Database Update:

- None. This is a read-only transaction.

Confirmation/Output:

- 200 OK with a list of evidence records, or a list of versions (each with version_no, file_name, file_hash, is_tamper_flagged, uploaded_at).

Screenshot:



Transaction 12: Download Evidence File

Input:

- evidence_id (path)
- version_id (path)

Validation:

- The user must be logged in and have evidence:view.
- The version must exist and belong to that evidence; otherwise 404 "Version not found".

Processing:

1. The system verifies the token and permission and loads the version.
2. It reads the stored file from GridFS using the version's storage reference.
3. It records the download in the audit log.
4. It sends the file back as a download with its original file name. (Integrity checking is a separate transaction, Transaction 13.)

Database Update:

- audit_transactions: +1 document with action evidence_downloaded and metadata version_id.
- The stored file and the version record are unchanged.

Confirmation/Output:

- 200 OK with the PDF file (content type application/pdf, original file name).

Screenshot:

Transaction 13: Verify Evidence Integrity (Tamper Detection)

Input:

- evidence_id (path)
- version_id (path)

Validation:

- The user must be logged in and have evidence:view.
- The version must exist and belong to that evidence; otherwise 404 "Version not found".

Processing:

1. The system loads the version and reads the stored file from GridFS.
2. It recomputes the file's SHA-256 hash.
3. It compares the new hash with the hash recorded when the file was uploaded.
4. If they differ and the version is not already flagged, it marks the version as tampered and writes an audit entry.

Database Update:

- If the hashes match: no change.
- If they differ: evidence_versions is updated (is_tamper_flagged = true) and audit_transactions gets +1 document with action tamper_flagged. The flag and the entry are written only the first time.

Confirmation/Output:

- 200 OK with version_id, intact (true or false) and is_tamper_flagged.

Screenshot:

Transaction 14: Delete Case (Soft Delete)

Input:

- case_id (path)

Validation:

- The user must be logged in and have case:delete; analysts and reviewers receive 403.
- The case must exist and must not already be deleted; otherwise 404 "Case not found".

Processing:

1. The system verifies the token and permission and loads the case.
2. It sets the deletion date (deleted_at) to the current time instead of removing the record, so the chain of custody is preserved.
3. It writes an audit entry.

Database Update:

- cases: the document is updated (deleted_at is set). It is not removed from the database.
- audit_transactions: +1 document with action case_deleted.
- Related evidence, versions and stored files are untouched.

Confirmation/Output:

- 204 No Content.
- The case no longer appears in GET /cases, and GET /cases/{case_id} returns 404.

Screenshot:



Transaction 15: Update User Account (Role / Active Status)

Input:

- user_id (path)

- role_id and/or is_active (JSON body, both optional)

Validation:

- The user must be logged in and have user:manage.
- The target user must exist; otherwise 404 "User not found".
- If role_id is given it must belong to an existing role; otherwise 404 "Role not found".

Processing:

1. The system verifies the token and permission and loads the target user.
2. It applies only the fields that were provided and saves the user.
3. It writes an audit entry listing what changed.

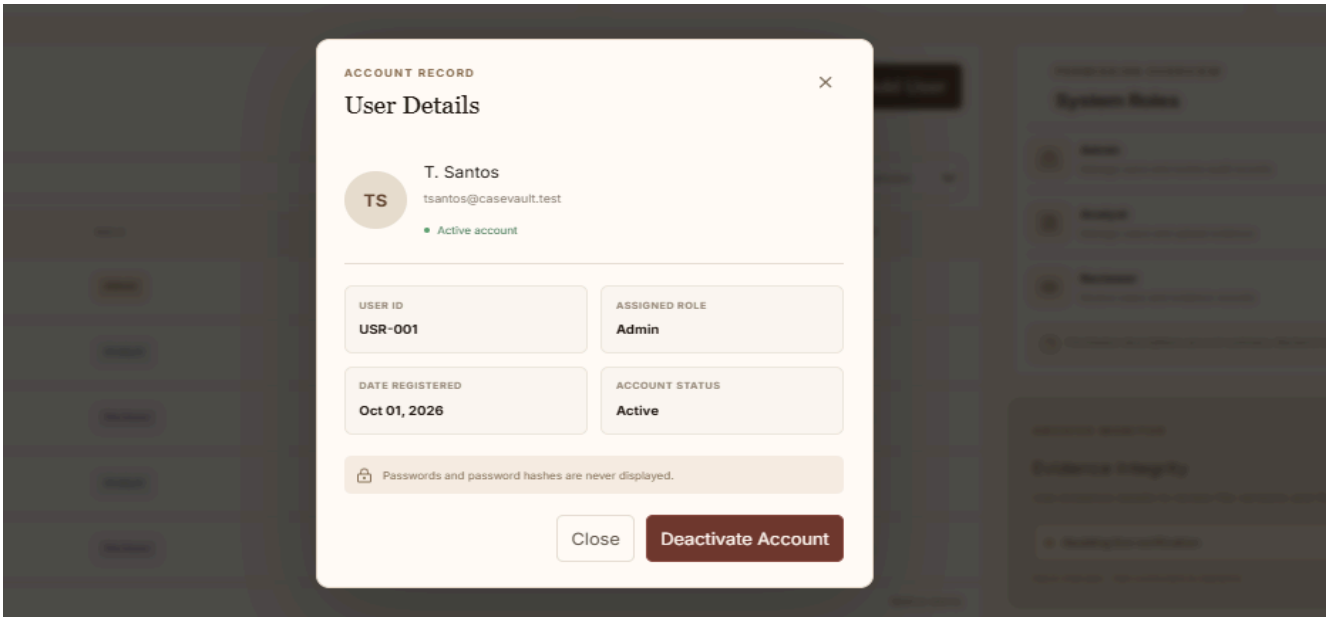
Database Update:

- users: the existing document is updated (role_id and/or is_active).
- audit_transactions: +1 document with action user_updated and metadata target and the changed fields.
- Accounts are deactivated, never deleted, so audit history stays intact.

Confirmation/Output:

- 200 OK with the updated user.
- A deactivated user can no longer log in in (403) and an existing token is rejected on the next request.

Screenshot:



Transaction 16: Change Own Password

Input:

- current_password
- new_password (JSON body)

Validation:

- The user must be logged in with an active account.
- current_password must match the stored hash; otherwise 400 "Current password is incorrect".
- new_password must be 8 to 128 characters (422).

Processing:

1. The system identifies the user from the token.
2. It checks the current password against the stored hash.
3. It hashes the new password with bcrypt and saves it.

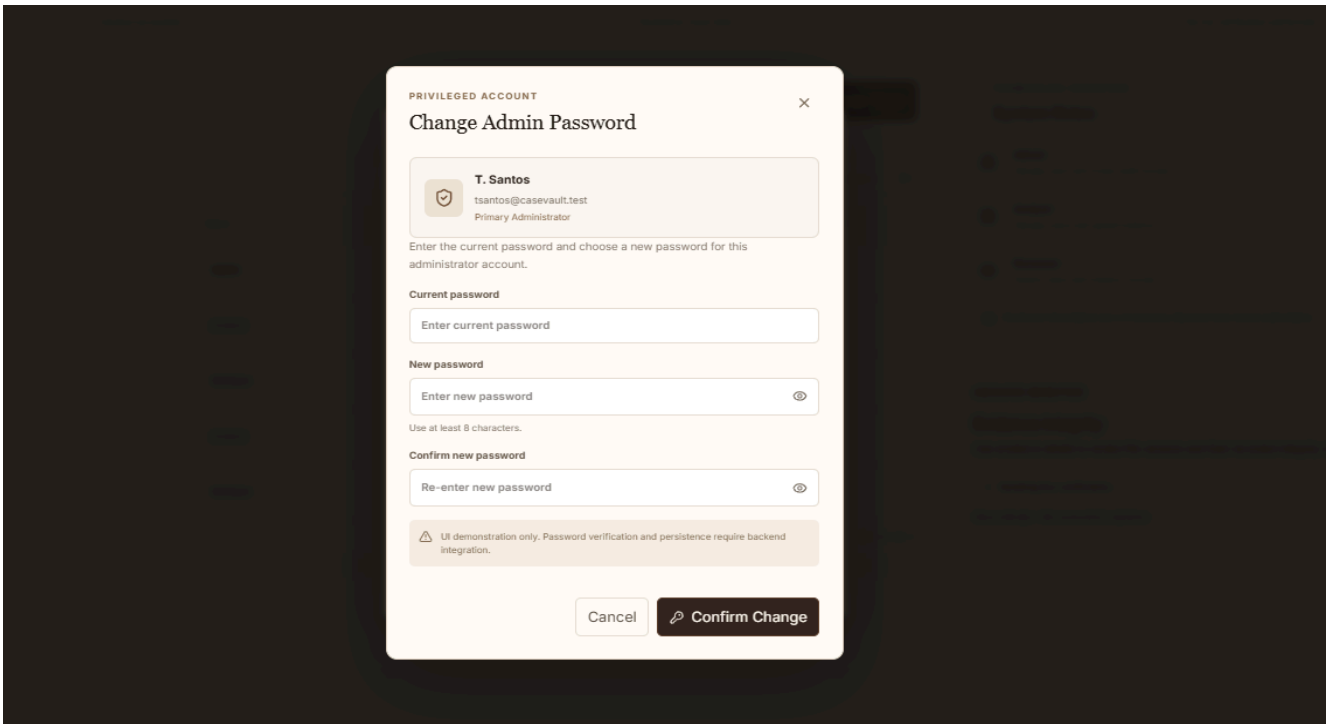
Database Update:

- users: hashed_password is replaced with the hash of the new password. The plain password is never stored.

Confirmation/Output:

- 204 No Content.
- The old password stops working at the next login.

Screenshot:



Transaction 17: View User Accounts and Roles

Input:

- No input other than the access token.

Validation:

- /users/me only requires a valid token and active account.
- /users and /roles require user:manage; other roles receive 403.
- Password hashes are never included in any response.

Processing:

1. /users/me returns the profile of the logged-in user.
2. /users lists all accounts.
3. /roles lists the roles so an admin can pick a role_id when creating or updating a user.

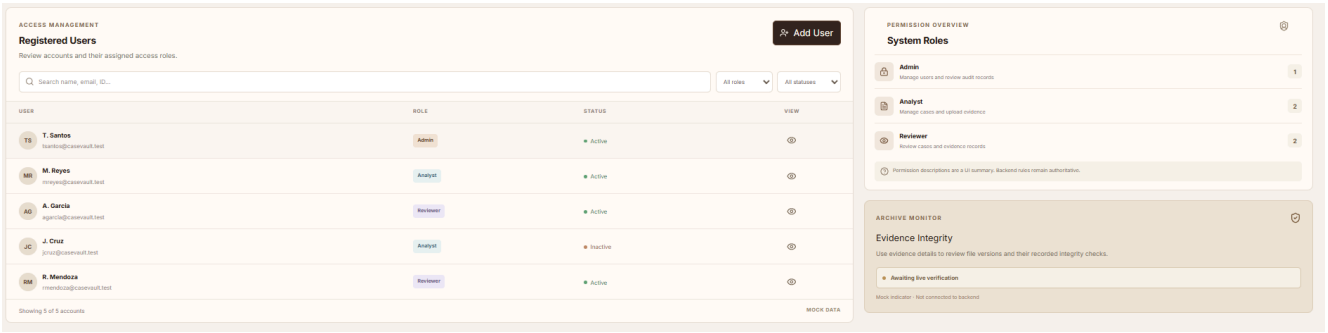
Database Update:

- None. These are read-only transactions.

Confirmation/Output:

- 200 OK with a profile or list of users (id, name, email, role_id, is_active, created_at, updated_at), or a list of roles (id, name, description).

Screenshot:



Transaction 18: View Audit Trail

Input:

- Optional filters: case_id, evidence_id, actor_user_id.
- Optional limit (default 100, maximum 500).

Validation:

- The user must be logged in and have user:manage; other roles receive 403.
- limit must not exceed 500 (422).

Processing:

1. The system verifies the token and permission.
2. It builds a filter from whichever parameters were supplied.

3. It fetches the matching entries, newest first, up to the limit.

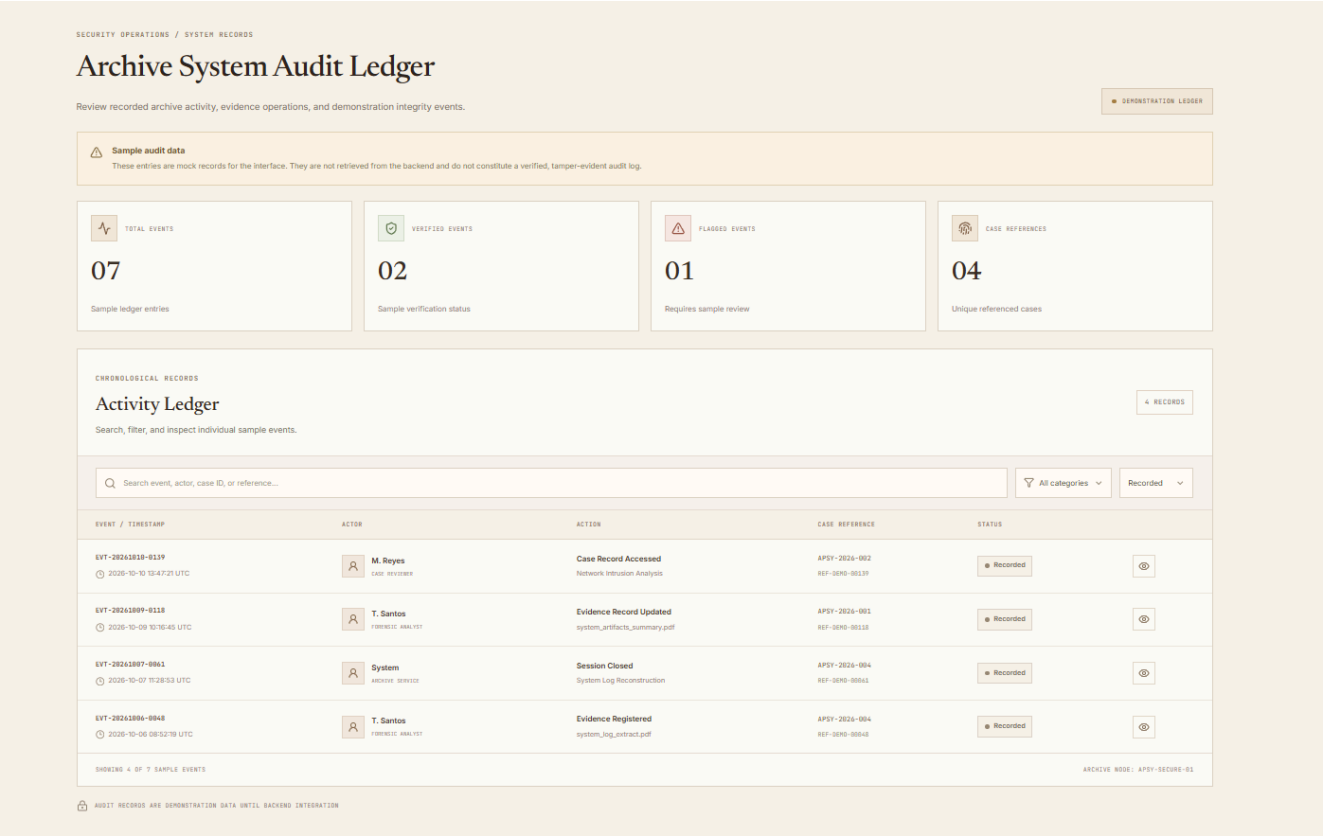
Database Update:

- None. The system never edits or deletes audit entries, so the log is append-only.

Confirmation/Output:

- 200 OK with a list of entries (id, actor_user_id, case_id, evidence_id, action, metadata, occurred_at).
- Filtering by case_id or evidence_id shows the chain of custody for that record.

Screenshot:



12. SOFTWARE TESTING

12.1 Test Plan

Case Vault's backend was tested through its REST API using functional (black-box) testing. Each test sent a request as a specific role (Administrator, Analyst, or Reviewer) and compared the response and the stored data with the expected result. The tests covered authentication, role-based access control, case management, evidence upload and versioning, tamper detection, and audit logging. Test files were PDFs, and tampering was simulated by changing a stored file's contents so its hash no longer matched.

12.2 Test Cases and Results

The Actual Result and Status columns are blank for you to fill in after running each case.

ID	Feature/Transaction	Expected Result	Actual Result	Status
TC-01	Login with valid credentials	An access token valid for 8 hours is returned, and user_logged_in is logged	An access token was returned, and *user_logged_in* appeared in the audit log.	Pass
TC-02	Login with a wrong password	401 error, no token	POST /auth/login returned HTTP 401: Unauthorized with “Invalid email or password”; no access token was returned.	Pass
TC-03	Login with a deactivated account	403 error, no token	Returned 403: Forbidden; **Response**: “This account has been deactivated"	Pass
TC-04	Access with an invalid or expired token	401 “Invalid or expired token”	Use of an invalid or expired token returned **HTTP 401: Unauthorized** **Response**: “"Invalid or expired token"	Pass
TC-05	Analyst tries a restricted action (e.g., delete a case)	403 Access denied	The use of an analyst account when deleting a case returned **HTTP 403: Forbidden;** **Response**: "You lack permission: case: delete”	Pass
TC-06	Reviewer tries to upload evidence	403 Access denied	The use of a reviewer account tries to upload evidence, returned **HTTP 403: Forbidden** **Response**: "You lack permission: case: create"	Pass
TC-07	Create a case with a unique	Case saved with an audit entry	Successful creation of a case returned **HTTP 201: Created**	Pass

	case number		**Response:** case_created appeared in the audit log	
TC-08	Create a case with a duplicate number	409 “A case with this number already exists”	Creating a case with a similar number was denied returned **HTTP 409: Conflict** **Response:** "A case with this number already exists"	Pass
TC-09	Upload a PDF to an active case	SHA-256 hash computed, file stored in GridFS, version and audit entry saved	PDF upload returned **HTTP 201: Created**. Evidence record and version 1 were created; the audit log shows evidence_uploaded with a SHA-256 file hash.	Pass
TC-10	Upload a non-PDF file, or upload to a deleted or missing case	Rejected (400 or 404)	Non-PDF file upload returned **HTTP 400: Bad Request** **Response:** "Only PDF files are accepted"	Pass
TC-11	Verify an unmodified evidence file	Reported as intact	Verified an unmodified evidence file **HTTP 200: OK** **Response:** "intact": true	Pass
TC-12	Verify a modified evidence file	Reported as not intact, the version is tamper-flagged, and *tamper_flagged* is logged	Verified a modified evidence file **HTTP 200: OK** **Response:** "intact": true	Pass
TC-13	Reviewer updates case status	Status saved and the action logged	Reviewer account successfully updated a cases status, **HTTP 200: OK** **Response:** "status": "closed",	Pass
TC-14	Administrator soft-deletes a case	The case is marked deleted, kept in the database, and hidden from the case list	Administrator account successfully soft-deletes a case, **HTTP 204: no content** **Response:** "detail": "Case not found" – but still present safely	Pass

			hidden inside the database itself only handled and accessed by the admin.	
TC-15	Administrator creates a user with an email already in use	409 “A user with this email already exists”	Creating a new user with an email already used is denied, **HTTP 409: Conflict** **Response:** "A user with this email already exists"	Pass

12.3 Defects and Corrective Actions

No defects were encountered during testing or development.

12.4 User Acceptance Testing (UAT)

UAT was not conducted.

13. CHALLENGES AND SOLUTIONS

Challenge	Solution
Data connection setup and intermittent failure	Startup initialization retries up to six attempts with three-second intervals; deployment documentation requires a transaction-capable MongoDB replica set.
Restricting access to sensitive operations	JWT authentication and role-permission dependencies protect API endpoints. The seed service defines admin, analyst, and reviewer grants.
Preserving uploaded file history and checking file integrity	Files are stored in GridFS, each evidence version stores a SHA-256 hash, and a verification endpoint compares a fresh hash with the saved value. Replacement uploads create new versions.

14. CONCLUSION

Case Vault's supplied backend implements core archive functions: authenticated API access, role-based permissions, case management, PDF evidence storage, evidence version records, SHA-256 integrity verification, case tags, case soft deletion, and audit records for selected actions. The implementation uses FastAPI, Beanie/Motor, MongoDB, and GridFS.

The objectives are partly achieved in the supplied backend. Case and evidence organization, access control, version records, and on-demand integrity checking are present. The backend does not provide all proposal capabilities: analyst/date search, comprehensive read/action logging, uniform atomic audit transactions, evidence deletion/archiving, a user-facing frontend, and continuous file monitoring are not implemented or evidenced. The backend does not enforce a case-status workflow. No test results, performance measurements, UI screenshots, or UAT evidence were supplied, so those outcomes cannot be evaluated in this report.

The next project work should be based on the team-approved scope and should prioritize completing agreed search and audit requirements, making audit updates atomic where required, strengthening file validation, resolving concurrent version allocation, and providing genuine test, UI, and UAT evidence for defense.

15. REFERENCES

1. Metaguard. *CCSFEN1L* – Project Documentation Format – Metaguard. Supplied PDF.
2. Metaguard. *CCSFEN1L* – Project Proposal: Case Vault: Forensic Log Analysis Documentation — A Secure Web-Based Archive for Analyzed Forensic Case Data. Supplied DOCX.
3. Metaguard. Case Vault Backend README. Supplied repository, README.md.
4. Metaguard. Case Vault Backend source code. Supplied repository, modules under app/.
5. Metaguard. Case Vault Backend dependency manifest. Supplied repository, requirements.txt.

16. APPENDICES

- **Appendix A** – Requirements Gathering Evidence: Supplied proposal, documentation format, README, dependency manifest, and source modules were reviewed. Interview, observation, questionnaire, and stakeholder-approval evidence was not supplied.
- **Appendix B** – Additional UML Diagrams: The sequence diagram is in Section 6.4. The use case, activity, class, architecture, and ER diagrams appear in Sections 6.1–6.3, 7, and 8.1.
- **Appendix C** – Additional Screenshots: No application screenshots were supplied. Add authentic screenshots from the implemented Case Vault frontend/API interface if available.
- **Appendix D** – Testing Evidence: No executed test reports, API captures, or test logs were supplied. Section 12.2 contains planned tests only.
- **Appendix E** – UAT Results: No UAT materials were supplied. Add completed session records and outcomes after user acceptance testing.
- **Appendix F** – Source Code / Repository Information: The supplied source is the case-vault-backend repository. Main entry point: app/main.py; routers: app/routers/; models: app/models/; schemas: app/schemas/; authentication dependencies: app/dependencies/; services: app/services/; core settings, database, and security: app/core/. The README documents running uvicorn app.main:app --reload and opening /docs. Do not include credentials or secret values in this report.
- **Appendix G** – Individual Contributions: Assigned responsibilities are taken from the supplied documentation format. Contribution statements are based on those roles and the project manager/backend developer clarification provided by the team. Confirm each member's completed work and attach available evidence before submission; the backend repository alone does not verify individual authorship or frontend, testing, and design activities.

INDIVIDUAL CONTRIBUTION

Member	Assigned Responsibilities	Actual Contribution
Berto, James A.	Project Manager / Backend Developer	Leads the project and team workflow, coordinates how system components are developed, and develops the

		backend. Backend responsibilities include the FastAPI routes and services, MongoDB/Beanie data models, authentication and role permissions, case and evidence operations, file versioning/integrity checks, and audit logging.
Salazar, Franz E.	Documentation	Organizes and prepares the project documentation, analyzes the project proposal and system behavior, describes requirements and features, and compiles diagrams and supporting materials for the documentation.
Sandova, Christian Ace E.	Tester / QA	Prepares and carries out functional and role-permission checks, checks case and evidence workflows including uploads, versioning, and integrity verification, records defects or unexpected results, and provides test evidence for the testing section.
Santos, Tyra B.	Initial Designer	Develops the initial interface concepts and layouts for the system, including the planned login, case/dashboard, evidence, and audit views, and

		provides the design direction to the frontend developer.
Wales, Cyralin Myles A.	Frontend Developer	Develops the user-facing frontend and connects its screens and user actions to the backend API, including authentication, case management, evidence handling, and audit-related views as supported by the agreed project scope.